Lecture 5. Unsupervised learning: structure without labels

Deep Learning for Scientific Discovery

2026/2027

1 Measurements without labels

A tumour removed during surgery can be ground up and its messenger RNA sequenced. The result is a list of numbers, one for every gene of the human genome, each stating how strongly that gene was transcribed in the tissue. For a few hundred patients this gives a table with a few hundred rows and about twenty thousand columns. A haematology laboratory photographs single blood cells under a microscope and stores each cell as an image of a few thousand pixel values. A bank stores every card transaction as a row of a few tens of numbers describing the payment. In all three cases the measurement is easy and plentiful, and in all three the scientist does not yet know which question the data will answer. Nobody has attached to each row the label that a supervised method would learn to predict, or the label exists only for a small part of the data, or the categories that matter have not yet been named.

The previous lectures treated supervised learning. There every sample $\mathbf{x}_i$ came with a target y_i, a digit class or a cell type, and a network was trained so that its output approximated y_i. The target defined the loss, and the loss defined what the network had to learn. Without targets only the samples remain,

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}, \qquad \mathbf{x}_i \in \mathbb{R}^D, \tag{1.1}$$

where n is the number of samples and D the number of measured quantities per sample, the dimension of the data. For the tumour table $\mathbf{x}_i$ is the expression profile of patient i and D is the number of genes. For a blood-cell image $\mathbf{x}_i$ lists the intensities of all pixels in all colour channels. It is convenient to stack the samples as the rows of the data matrix

$$X = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix} \in \mathbb{R}^{n \times D}, \qquad X_{ij} = x_{ij} = \text{feature } j \text{ of sample } i. \tag{1.2}$$

Figure 1 shows this arrangement for the tumour data, where a row is a patient and a column a gene.

Without targets, the questions put to such data are of a different kind. A scientist wants to know whether a few combinations of the measured quantities describe most of the variation between samples, whether the samples fall into groups, and which samples are unusual. The first question asks for a small number of underlying factors. In the tumour table it asks whether the twenty thousand expression values of a patient are governed by a handful of biological programs, sets of genes that are switched on and off together. The second asks for a classification that nobody has supplied, for instance whether tumours of one organ form one group or several. The third asks for a notion of typical and atypical, so that a

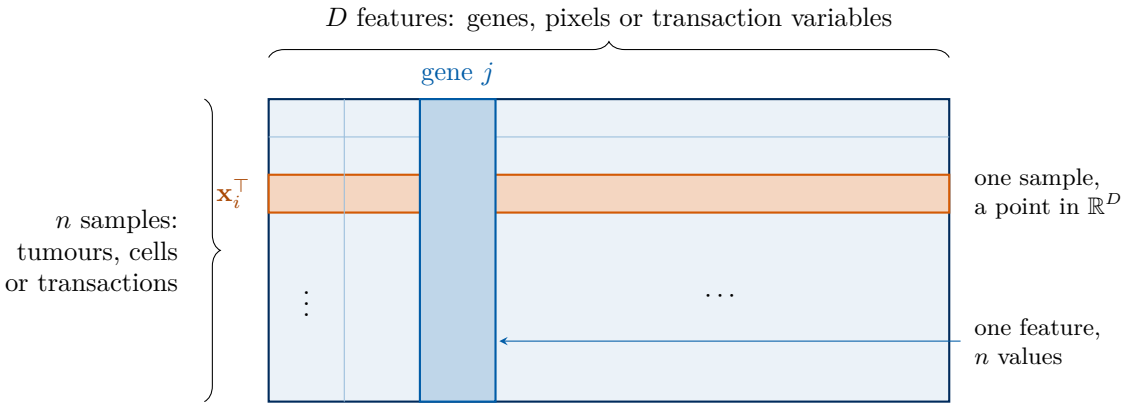


Figure 1: The data matrix X of equation (1.2). Each row is one sample $\mathbf{x}_i^\top$, a point in the D-dimensional feature space, and each column collects one measured quantity over all samples. For the tumour data a row is the expression profile of one patient and a column the expression of one gene in all patients.

fraudulent payment or a cell of a kind never seen before can be flagged. These questions are answered respectively by dimensionality reduction, by clustering and by anomaly detection, and together they form the subject of unsupervised learning treated in this chapter.

In probabilistic language the samples are independent draws from an unknown distribution $p(\mathbf{x})$ over $\mathbb{R}^D$. Supervised learning estimates the conditional distribution $p(y \mid \mathbf{x})$ of a target given a sample. Unsupervised learning asks about $p(\mathbf{x})$ itself. A low-dimensional description says that $p(\mathbf{x})$ is concentrated near a set of dimension much smaller than D. Clusters are regions of high density separated by regions of low density, and an anomaly is a sample in a region where $p(\mathbf{x})$ is small. None of the methods of this chapter estimates $p(\mathbf{x})$ directly. Each of them extracts one feature of it, and the closing section returns to what is lost by not having $p(\mathbf{x})$ itself.

1.1 Why structure is hard to find

The first difficulty that separates this problem from the supervised one is that the data cannot be looked at. A table of 801 rows and 20 531 columns, or a set of 3421 images with 2352 numbers each, has no picture. A scatter plot shows two coordinates at a time, and with twenty thousand coordinates there are about two hundred million pairs of them. A method is needed that produces a picture, and the picture then shows what the method has kept and not necessarily what the data contain.

The second difficulty is that there is no target to optimise. In supervised learning the quality of a model is measured by its error on held-out targets, and two models can be compared by that error. Without targets every method must define its own criterion of what structure is, for example a large variance, a preserved neighbourhood, a small distance to a cluster centre or a good reconstruction. Different criteria reveal different structures in the same data, and no single number tells which of them is correct. Labels, when they exist for part of the data, are therefore used only after a method has run, to judge whether the structure it found corresponds to a known distinction. Throughout the chapter labels serve only this evaluation and the colouring of plots.

The third difficulty is geometric. Most methods rely on distances between samples, and in high dimension distances behave in a way that low-dimensional intuition does not anticipate.

Consider two independent random points $\mathbf{x}$ and $\mathbf{x}'$ whose D coordinates are independent and normally distributed with mean zero and variance η^2. The difference of one coordinate, $x_j - x'_j$, is normal with variance $2\eta^2$, so its square has the mean $2\eta^2$ and, from the fourth moment $3(2\eta^2)^2$ of a normal variable, the variance $12\eta^4 - 4\eta^4 = 8\eta^4$. The squared distance is a sum of D independent such terms, hence

$$\mathbb{E}\,\|\mathbf{x}-\mathbf{x}'\|^2 = 2D\eta^2, \qquad \mathrm{Var}\|\mathbf{x}-\mathbf{x}'\|^2 = 8D\eta^4, \qquad \frac{\sqrt{\mathrm{Var}\|\mathbf{x}-\mathbf{x}'\|^2}}{\mathbb{E}\,\|\mathbf{x}-\mathbf{x}'\|^2} = \sqrt{\frac{2}{D}}. \tag{1.3}$$

The relative spread of the squared distances falls like $1/\sqrt{D}$, and the distance itself, by the rule $\delta(r)/r \approx \frac{1}{2}\,\delta(r^2)/r^2$ for small relative errors, has the relative spread $1/\sqrt{2D}$. For $D = 1000$ the distances of typical pairs of points differ from their mean by about two percent, the relative standard deviation $1/\sqrt{2000}$. Figure 2 shows the distribution of the distance, scaled by its typical value $\sqrt{2D}\,\eta$, for four dimensions. For two dimensions the distances range from zero to about twice the typical value, while for a thousand dimensions they form a narrow peak. In such a cloud the nearest neighbour of a point is hardly closer than its farthest point, and the statement that two samples are close carries almost no information [1].

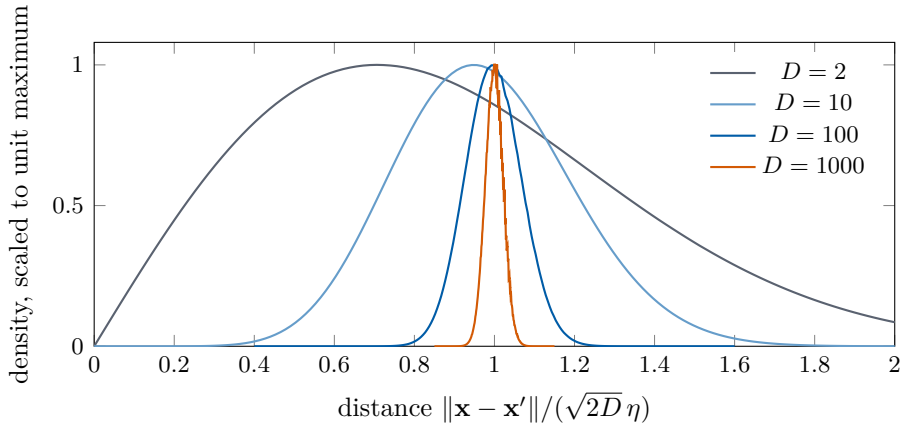


Figure 2: Concentration of distances. For two random points with independent normal coordinates, the distance divided by its typical value $\sqrt{2D}\,\eta$ is distributed as $\chi_D/\sqrt{D}$, where χ_D is a chi-distributed variable with D degrees of freedom, the length of a vector of D independent standard normal numbers. Each curve is scaled to the same maximum for comparison. As D grows the distribution narrows around one, with relative width $1/\sqrt{2D}$, equation (1.3), so that all pairs of points become nearly equidistant.

Real data escape this fate only because they are not random points of $\mathbb{R}^D$. If every pixel of a 28×28 image is set to an independent random grey value, the result is noise and never a handwritten digit. Images of digits, of cells or of anything else occupy a very thin part of pixel space, in which neighbouring pixels are strongly correlated and only a few independent changes, such as the thickness, the slant or the size of a stroke, move an image from one admissible picture to another. The working assumption of this chapter, often called the manifold hypothesis, is that the data lie close to a set of dimension much smaller than D, possibly curved, and that the structure of interest is the shape of that set and the way the samples are distributed on it. Each method of the chapter makes a different assumption about the shape of that set, and the chapter proceeds from the simplest assumption, a flat subspace, to more flexible ones.

1.2 Four data sets

Four data sets are carried through the chapter. One of them is a test bed with known answers, on which every method can be checked, and three are scientific data sets on which the methods are applied. Table 1 lists their sizes.

The test bed is the MNIST collection of handwritten digits [2]. It contains 70 000 greyscale images, 60 000 for training and 10 000 for testing, each of 28×28 pixels with grey levels from 0 (background) to 255 (ink), which we divide by 255 so that every pixel lies in $[0, 1]$. One sample is therefore a vector $\mathbf{x} \in [0, 1]^{784}$. Each image carries a label from 0 to 9, and the ten digits are about equally frequent. Because we know that the data consist of ten kinds of objects, MNIST shows whether a method can find such kinds without being told about them. For maps and clusterings we use 5000 randomly chosen test images. Figure 3 shows examples.

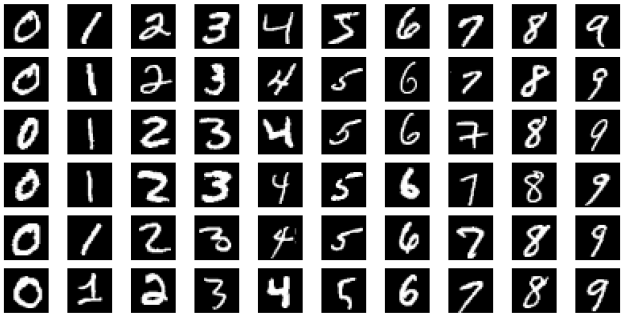


Figure 3: Examples of MNIST handwritten digits. Each image has 28×28 grey values, so that one sample is a point of $\mathbb{R}^{784}$.

The first scientific data set consists of microscope images of single blood cells from the BloodMNIST collection [3], which is derived from a data set of peripheral blood smears of individuals without infection and without haematological or oncological disease [4]. Each image is 28×28 pixels with three colour channels, so that $D = 28 \cdot 28 \cdot 3 = 2352$, and each cell has been assigned by clinical pathologists to one of eight types. Six of them are white blood cells (basophil, eosinophil, neutrophil, lymphocyte, monocyte and immature granulocyte, a white cell that is not yet mature), the erythroblast is an immature red blood cell, and the platelet is a small cell fragment that takes part in clotting. Of the 17 092 images, 11 959 form the training part, 1712 the validation part and 3421 the test part. White blood cells are distinguished by the shape of the nucleus and by the granules of the cytoplasm, features that occupy a few pixels at this resolution. Figure 4 shows three test images of each type.

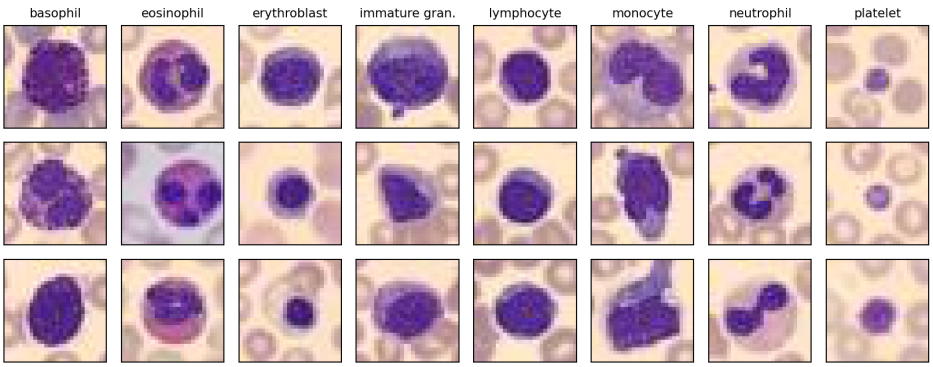


Figure 4: Three test images of each of the eight types of BloodMNIST cells, 28×28 pixels in three colours. The test part holds between 243 and 666 images per type.

The second scientific data set records 284 807 card transactions made by European cardholders on two days of September 2013, of which 492 are fraudulent [5]. To protect the cardholders, the bank released the original variables of each transaction only after transforming them by principal component analysis, the method of Section 2, into 28 columns $V_1, \dots, V_{28}$ that have no readable meaning. Only the amount in euros and the time since the first transaction are original. One sample is

$$\mathbf{x}_i = (V_1, \dots, V_{28}, \ln(1 + \text{amount})) \in \mathbb{R}^{29}, \tag{1.4}$$

where the logarithm compresses the wide range of amounts, and the time is not used. Each transaction carries a label, fraud or normal, and the methods are applied as if fraud had never been observed before.

The third scientific data set is a random extraction of 801 tumour samples from the pan-cancer RNA sequencing data of The Cancer Genome Atlas [6, 7]. Each sample is a vector of 20 531 gene-expression values, with values between 0 and about 20.8, which appear to be on a logarithmic scale, although the normalisation is not documented by the provider. The gene names are anonymised, so that a column is known only by its index. Each sample belongs to one of five tumour types, breast invasive carcinoma (BRCA, 300 samples), kidney renal clear cell carcinoma (KIRC, 146), lung adenocarcinoma (LUAD, 141), prostate adenocarcinoma (PRAD, 136) and colon adenocarcinoma (COAD, 78). This data set has $D \gg n$, about twenty-five times more measured quantities than samples, a situation typical of genomics.

Table 1: The data sets of the chapter. n counts the samples, D the measured quantities per sample. The labels are used only to evaluate the unsupervised methods.

data set	one sample	n	D	label (evaluation only)
MNIST digits	28×28 grey image	70 000	784	digit 0–9
BloodMNIST	28×28 colour image	17 092	2352	cell type, 8 types
card transactions	28 principal components and log amount	284 807	29	fraud or normal
tumour expression	expression of 20 531 genes	801	20 531	tumour type, 5 types

The three scientific data sets pose the three questions of unsupervised learning in different proportions. For the blood cells the question is whether the eight cell types appear as groups when the images are mapped without labels. For the transactions the question is whether fraud can be recognised as unusual from normal transactions alone. For the tumours the question is whether a tumour type never seen before is recognised as different from the known types. Each of these questions requires a representation of the data in which the relevant structure is visible, and the simplest such representation is a linear projection onto a few well chosen directions.

2 Principal component analysis

2.1 Variance along a direction

The simplest low-dimensional description of a cloud of points is a flat one. If the samples lie close to a d-dimensional plane through their mean, each of them is described, up to a small error, by its d coordinates in that plane. The question is which plane to choose. Principal component analysis (PCA), introduced by Pearson as the problem of the plane of closest fit [8] and by Hotelling as the search for a few uncorrelated variables that carry the variation

of many [9], answers it in two equivalent ways. The plane is the one along which the data vary most, and it is also the one from which the data deviate least. We derive both answers and show that they coincide.

All quantities are measured from the mean of the data,

$$\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i, \tag{2.1}$$

which for MNIST is the average image. How two features vary together over the data set is described by the covariance matrix, the $D \times D$ matrix with entries

$$C_{jl} = \frac{1}{n}\sum_{i=1}^{n} (x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l), \qquad j, l = 1, \ldots, D. \tag{2.2}$$

The diagonal entry C_{jj} is the variance of feature j. An off-diagonal entry is positive when features j and l tend to be large in the same samples and negative when one tends to be large where the other is small. If the centred data matrix X_c has the rows $(\mathbf{x}_i - \bar{\mathbf{x}})^\top$, equation (2.2) reads

$$C = \frac{1}{n} X_c^\top X_c. \tag{2.3}$$

C is symmetric, and it is positive semidefinite, since for any vector $\mathbf{u}$ the number $\mathbf{u}^\top C \mathbf{u} = \frac{1}{n}\|X_c \mathbf{u}\|^2$ cannot be negative. If each feature is first divided by its standard deviation, the same formula gives the correlation matrix, which has ones on its diagonal. The choice between the two matters when the features are measured in different units or have very different spreads, a point that returns for the gene-expression data.

Consider a unit vector $\mathbf{u} \in \mathbb{R}^D$, $\mathbf{u}^\top \mathbf{u} = 1$, and the coordinate of each centred sample along it, $t_i = \mathbf{u}^\top (\mathbf{x}_i - \bar{\mathbf{x}})$. The coordinates t_i have mean zero, and their variance is

$$\frac{1}{n}\sum_{i=1}^{n} t_i^2 = \frac{1}{n}\sum_{i=1}^{n} \mathbf{u}^\top (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \mathbf{u} = \mathbf{u}^\top C\, \mathbf{u}. \tag{2.4}$$

The covariance matrix thus contains the variance of the data along every direction. The direction of largest variance maximises $\mathbf{u}^\top C \mathbf{u}$ under the constraint $\mathbf{u}^\top \mathbf{u} = 1$. With a Lagrange multiplier λ for the constraint, the stationarity condition of $\mathbf{u}^\top C \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1)$ is

$$\frac{\partial}{\partial \mathbf{u}} \Big[\mathbf{u}^\top C \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1) \Big] = 2C\mathbf{u} - 2\lambda \mathbf{u} = 0 \quad \Longrightarrow \quad C\mathbf{u} = \lambda \mathbf{u}. \tag{2.5}$$

A stationary direction is an eigenvector of C, and at such a direction the variance (2.4) equals $\mathbf{u}^\top C \mathbf{u} = \lambda \mathbf{u}^\top \mathbf{u} = \lambda$, its eigenvalue. The variance is therefore largest along the eigenvector with the largest eigenvalue.

Since C is real and symmetric, it has D real eigenvalues and an orthonormal basis of eigenvectors. We order them by decreasing eigenvalue,

$$C\mathbf{v}_l = \lambda_l \mathbf{v}_l, \qquad \mathbf{v}_l^\top \mathbf{v}_m = \delta_{lm}, \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0, \tag{2.6}$$

where δ_{lm} is the Kronecker delta. Collecting the eigenvectors as columns of the orthogonal matrix $V = (\mathbf{v}_1, \ldots, \mathbf{v}_D)$ gives the spectral decomposition

$$C = V \Lambda V^\top, \qquad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_D). \tag{2.7}$$

Definition 2.1 (Principal components)**.** The principal directions of the data are the eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \ldots$ of the covariance matrix (2.2), ordered by decreasing eigenvalue. The l-th principal component of sample i, also called its score, is its coordinate along $\mathbf{v}_l$,

$$w_i^{(l)} = \mathbf{v}_l^\top (\mathbf{x}_i - \bar{\mathbf{x}}). \tag{2.8}$$

Keeping the first d directions, collected in the $D \times d$ matrix $V_d = (\mathbf{v}_1, \ldots, \mathbf{v}_d)$, gives the d-dimensional representation and the reconstruction

$$\mathbf{w}_i = V_d^\top (\mathbf{x}_i - \bar{\mathbf{x}}) \in \mathbb{R}^d, \qquad \hat{\mathbf{x}}_i = \bar{\mathbf{x}} + V_d\, \mathbf{w}_i \in \mathbb{R}^D. \tag{2.9}$$

The entries of $\mathbf{v}_l$ are called the loadings of the features on component l.

The second direction follows from the same argument restricted to directions orthogonal to $\mathbf{v}_1$. Any unit vector orthogonal to $\mathbf{v}_1$ can be written as $\mathbf{u} = \sum_{l \geq 2} c_l \mathbf{v}_l$ with $\sum_{l \geq 2} c_l^2 = 1$, and its variance is $\mathbf{u}^\top C \mathbf{u} = \sum_{l \geq 2} \lambda_l c_l^2 \leq \lambda_2$, with equality for $\mathbf{u} = \mathbf{v}_2$. The scores along different eigenvectors are uncorrelated, since $\frac{1}{n} \sum_i w_i^{(l)} w_i^{(m)} = \mathbf{v}_l^\top C \mathbf{v}_m = \lambda_m \delta_{lm}$. PCA therefore replaces the D correlated features by D uncorrelated ones, ordered by their variance, and dimensionality reduction consists in keeping the first d of them.

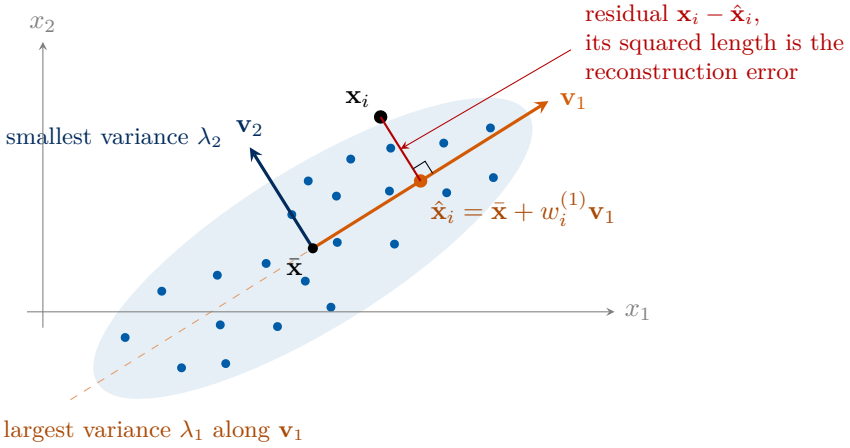


Figure 5: PCA as a rotation of the coordinate system. The principal directions $\mathbf{v}_1$ and $\mathbf{v}_2$ are the eigenvectors of the covariance matrix and point along the axes of the cloud, $\mathbf{v}_1$ along the largest spread. A sample $\mathbf{x}_i$ is projected onto $\mathbf{v}_1$ (orange point), which gives its score $w_i^{(1)}$ and its one-component reconstruction $\hat{\mathbf{x}}_i$. The perpendicular residual (red) is the part of the sample that the projection loses. Maximising the spread of the orange points and minimising the total squared length of the red segments are the same task, equation (2.10).

2.2 Minimal reconstruction error

The second view of PCA asks for the d-dimensional plane through $\bar{\mathbf{x}}$ from which the data deviate least. Let the plane be spanned by the orthonormal columns of a $D \times d$ matrix R, $R^\top R = \mathbb{1}_d$, where $\mathbb{1}_d$ is the identity matrix. The point of the plane closest to $\mathbf{x}_i$ is the orthogonal projection $\bar{\mathbf{x}} + RR^\top (\mathbf{x}_i - \bar{\mathbf{x}})$, and the residual $(\mathbb{1} - RR^\top)(\mathbf{x}_i - \bar{\mathbf{x}})$ is perpendicular to the plane. Pythagoras' theorem splits the squared length of the centred sample into the part in the plane and the residual,

$$\|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \|R^\top (\mathbf{x}_i - \bar{\mathbf{x}})\|^2 + \|(\mathbb{1} - RR^\top)(\mathbf{x}_i - \bar{\mathbf{x}})\|^2. \tag{2.10}$$

Averaging over the samples and using equation (2.4) for each column of R, the mean squared reconstruction error is

$$\mathcal{E}(R) = \frac{1}{n}\sum_{i=1}^{n} \|(\mathbb{1} - RR^{\top})(\mathbf{x}_i - \bar{\mathbf{x}})\|^2 = \operatorname{tr} C - \operatorname{tr}\left(R^{\top} C R\right). \tag{2.11}$$

The first term comes from $\frac{1}{n}\sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \sum_j C_{jj} = \operatorname{tr} C$. It is the total variance and does not depend on the plane. Minimising the reconstruction error is therefore the same as maximising $\operatorname{tr}(R^{\top} C R)$, the variance captured in the plane. The two views of PCA, largest variance and smallest error, are one optimisation problem. It remains to show that the maximum is attained for the principal directions. Write each column $\mathbf{r}_a$ of R in the eigenbasis of C,

$$\mathbf{r}_a = \sum_{l=1}^{D} (\mathbf{v}_l^{\top}\mathbf{r}_a)\mathbf{v}_l \quad \Longrightarrow \quad \mathbf{r}_a^{\top} C \mathbf{r}_a = \sum_{l=1}^{D} \lambda_l (\mathbf{v}_l^{\top}\mathbf{r}_a)^2, \tag{2.12}$$

where the second form follows from $C\mathbf{v}_l = \lambda_l \mathbf{v}_l$ and the orthonormality of the eigenvectors. Summing over the d columns gives

$$\operatorname{tr}\left(R^{\top} C R\right) = \sum_{a=1}^{d} \mathbf{r}_a^{\top} C \mathbf{r}_a = \sum_{l=1}^{D} \lambda_l \, c_l, \qquad c_l = \sum_{a=1}^{d} \left(\mathbf{v}_l^{\top}\mathbf{r}_a\right)^2 = \|R^{\top}\mathbf{v}_l\|^2. \tag{2.13}$$

Each weight c_l is the squared length of the projection of the unit vector $\mathbf{v}_l$ onto the plane, so $0 \le c_l \le 1$, and their sum is $\sum_l c_l = \operatorname{tr}(R^{\top}R) = d$. A sum $\sum_l \lambda_l c_l$ with decreasing λ_l and weights between 0 and 1 that add up to d is largest when the weight 1 sits on the d largest eigenvalues. The maximum, $\lambda_1 + \cdots + \lambda_d$, is reached for $R = V_d$, and the minimal reconstruction error is the sum of the discarded eigenvalues,

$$\min_{R} \mathcal{E}(R) = \mathcal{E}(V_d) = \sum_{l=d+1}^{D} \lambda_l. \tag{2.14}$$

The plane is unique when $\lambda_d > \lambda_{d+1}$. Within the plane any orthonormal basis gives the same error, so the error alone fixes the subspace and the variance ordering fixes the individual directions inside it.

The fraction of the total variance retained by d components,

$$r_d = \frac{\lambda_1 + \cdots + \lambda_d}{\lambda_1 + \cdots + \lambda_D} = 1 - \frac{\mathcal{E}(V_d)}{\operatorname{tr} C}, \tag{2.15}$$

is the explained-variance ratio, and $1 - r_d$ is the reconstruction error measured in units of the total variance. A plot of $\lambda_l / \operatorname{tr} C$ against l, the spectrum of the data, shows how quickly the variance is exhausted. A spectrum that falls steeply to a small number of dominant eigenvalues indicates a nearly flat low-dimensional structure, and a flat spectrum indicates that no plane of small dimension describes the data.

A two-dimensional example shows the construction. For 400 points drawn from a correlated normal distribution in the plane, the two principal axes are the axes of the elliptical cloud, as in Figure 5, and the first of them carries 94.5% of the variance. Describing each point by its single coordinate along $\mathbf{v}_1$ loses only the remaining 5.5%.

2.3 The singular value decomposition

In practice the eigenvectors are computed from the singular value decomposition (SVD) of the centred data matrix, without forming the covariance matrix. Every real $n \times D$ matrix can

be written as

$$X_c = USV^\top = \sum_{l=1}^{m} s_l \, \mathbf{u}_l \mathbf{v}_l^\top, \qquad m = \min(n, D), \tag{2.16}$$

where $U = (\mathbf{u}_1, \ldots, \mathbf{u}_m)$ has orthonormal columns $\mathbf{u}_l \in \mathbb{R}^n$, $V = (\mathbf{v}_1, \ldots, \mathbf{v}_m)$ has orthonormal columns $\mathbf{v}_l \in \mathbb{R}^D$, and $S = \operatorname{diag}(s_1, \ldots, s_m)$ holds the singular values $s_1 \geq s_2 \geq \cdots \geq 0$. Each term $s_l \mathbf{u}_l \mathbf{v}_l^\top$ is a matrix of rank one. Inserting the decomposition into equation (2.3) gives

$$C = \frac{1}{n} V S U^\top U S V^\top = V \, \frac{S^2}{n} \, V^\top, \qquad \lambda_l = \frac{s_l^2}{n}. \tag{2.17}$$

The right singular vectors are the principal directions, the squared singular values divided by n are the variances, and the scores of all samples follow without any further projection,

$$X_c V = US, \qquad w_i^{(l)} = s_l \, U_{il}. \tag{2.18}$$

Figure 6 shows the shapes of the factors and the truncation to d terms. The SVD avoids forming C. The condition number of the data matrix, the ratio s_1/s_m of its largest to its smallest singular value, measures how strongly rounding errors are amplified, and forming C squares it, so that eigenvalues smaller than the largest one by a factor near the rounding precision are lost. The SVD is how numerical libraries compute PCA. On the 60 000 MNIST training images the eigenvectors obtained by diagonalising C and those obtained from the SVD agree to floating-point rounding once the arbitrary sign of each eigenvector is fixed. The sign is arbitrary because $-\mathbf{v}_l$ is an eigenvector whenever $\mathbf{v}_l$ is one.

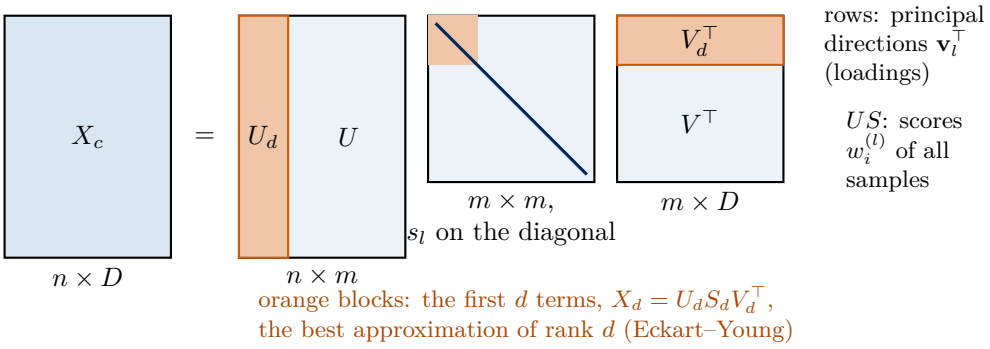


Figure 6: The singular value decomposition (2.16) of the centred data matrix. The columns of V are the principal directions, the singular values give the variances $\lambda_l = s_l^2/n$, and US contains the scores. Keeping the first d columns of U and V and the first d singular values (orange) gives the rank-d matrix X_d, whose rows are the PCA reconstructions of the samples.

The truncated decomposition $X_d = \sum_{l \leq d} s_l \mathbf{u}_l \mathbf{v}_l^\top$ has the rows $(\hat{\mathbf{x}}_i - \bar{\mathbf{x}})^\top$ of equation (2.9). The derivation of Section 2.2 showed that no projection onto another d-dimensional plane does better. A stronger statement holds, and it is used again for autoencoders in Section 5.3.

Eckart–Young theorem. *Let $F \in \mathbb{R}^{n \times D}$ be any matrix with the singular value decomposition $F = \sum_l \varsigma_l \, \mathbf{e}_l \mathbf{f}_l^\top$, singular values $\varsigma_1 \geq \varsigma_2 \geq \cdots \geq 0$, orthonormal left singular vectors $\mathbf{e}_l$ and orthonormal right singular vectors $\mathbf{f}_l$, and let $F_d = \sum_{l \leq d} \varsigma_l \, \mathbf{e}_l \mathbf{f}_l^\top$. For every matrix $Y \in \mathbb{R}^{n \times D}$ of rank at most d,*

$$\|F - Y\|_F^2 \;\geq\; \|F - F_d\|_F^2 = \sum_{l > d} \varsigma_l^2, \tag{2.19}$$

where $\|A\|_F^2 = \sum_{ij} A_{ij}^2$ is the squared Frobenius norm of a matrix A. If $\varsigma_d > \varsigma_{d+1}$, the minimiser F_d is unique [10].

For $F = X_c$ the singular values are the s_l of equation (2.16), $F_d = X_d$, and the theorem states that the truncated SVD is the best approximation of the centred data by a matrix of rank d. The theorem follows from the derivation of Section 2.2 in two steps. A matrix Y of rank at most d has its rows in some subspace of dimension at most d, spanned by the orthonormal columns of a matrix R. For fixed R the row of Y closest to the i-th row of F is its orthogonal projection, so $\|F - Y\|_F^2 \geq \|F - FRR^\top\|_F^2$. The derivation of equations (2.11) to (2.14) used only that $nC = X_c^\top X_c$ is a symmetric positive semidefinite matrix with the orthonormal eigenvectors $\mathbf{v}_l$, and nothing about centring. The same steps applied to $F^\top F$, whose eigenvalues are ς_l^2 and whose eigenvectors are $\mathbf{f}_l$, give

$$\|F - FRR^\top\|_F^2 = \operatorname{tr}(F^\top F) - \operatorname{tr}(R^\top F^\top F R) \;\geq\; \sum_l \varsigma_l^2 - \sum_{l \leq d} \varsigma_l^2 = \sum_{l > d} \varsigma_l^2, \tag{2.20}$$

which is the bound (2.19). The theorem therefore says more than the optimality of PCA among projections. No matrix of rank d, however it is constructed, approximates the centred data better than the truncated SVD. Compression of the data into d numbers per sample by any linear map, followed by any linear reconstruction, cannot beat PCA.

2.4 PCA of handwritten digits

We apply the construction to the 60 000 MNIST training images. The mean image $\bar{\mathbf{x}}$, the 784×784 covariance matrix C, its spectrum and the first four eigenvectors are shown in Figure 7. The covariance matrix has a banded structure, because neighbouring pixels in a row are strongly correlated, and the side bands correspond to pixels in neighbouring rows, 28 positions apart in the flattened vector. Pixels near the corners are background in practically every image, so their rows and columns of C are close to zero and they contribute eigenvalues close to zero.

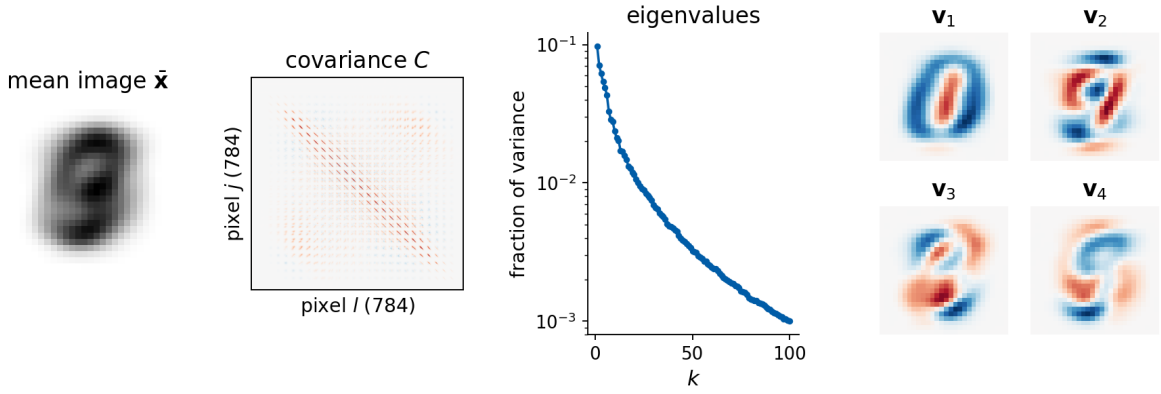


Figure 7: PCA of the 60 000 MNIST training images. From left to right, the mean image $\bar{\mathbf{x}}$, the 784×784 covariance matrix C (red positive, blue negative), the first 100 eigenvalues divided by their sum on a logarithmic scale, and the first four principal directions $\mathbf{v}_1, \ldots, \mathbf{v}_4$ displayed as 28×28 images, red for positive and blue for negative loadings.

The spectrum falls quickly over the first ten components and slowly afterwards. Table 2 lists the explained-variance ratio (2.15). Two components retain 16.8% of the variance, and 87 are needed for 90%. The digits are therefore far from lying in a plane of small dimension. A few dozen directions carry most of their variation, but none of the first few carries a dominant share.

The loadings have a direct interpretation for images, because each principal direction is itself an image. The score $w^{(1)} = \mathbf{v}_1^\top(\mathbf{x} - \bar{\mathbf{x}}) = \sum_j v_{1j}(x_j - \bar{x}_j)$ is a weighted sum of pixel deviations

Table 2: Explained-variance ratio r_d of PCA on the 60 000 MNIST training images.

number of components d	2	3	10	50	87
variance retained r_d	0.168	0.230	0.488	0.825	≥ 0.90

from the mean image, and it is large when ink appears where $\mathbf{v}_1$ is positive and is absent where $\mathbf{v}_1$ is negative. In Figure 7 the first direction has a vertical stroke of one sign in the centre and a ring of the opposite sign around it. Its score therefore contrasts images with ink on a ring, such as a 0, with images with ink only along a central vertical line, such as a 1. The second direction shows a diagonal pattern, and the later directions show increasingly fine patterns. A principal direction is a pattern of pixels that brighten and darken together across the data set. It need not resemble any single digit, since it describes how digits differ from the mean image.

Projecting the 5000 test images onto the first two directions gives the map of Figure 8. Without colours a single cloud appears, with a dense lobe on one side. Colouring by the true digit shows that the lobe consists of 1s, as expected from the interpretation of $\mathbf{v}_1$, and that the 0s lie on the opposite side, while the other eight digits overlap in the middle. A third component, which raises the retained variance from 16.8% to 23.0%, separates a few more digits but leaves most of them overlapping.

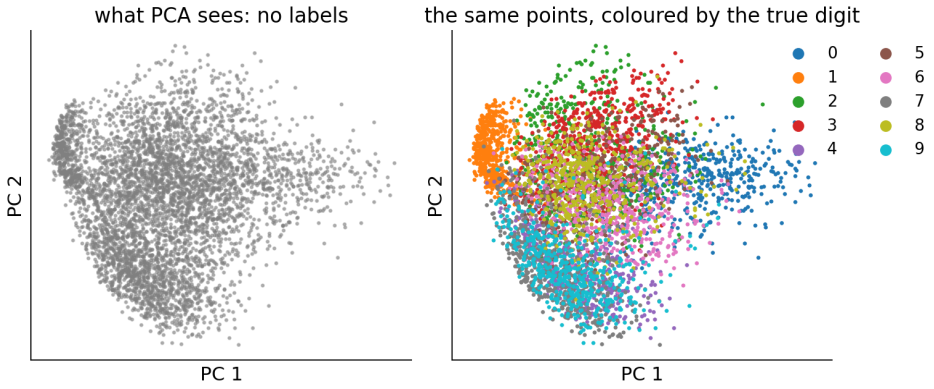


Figure 8: 5000 MNIST test images projected onto the first two principal directions of the training images. Left, the projection as PCA produces it, without labels. Right, the same points coloured by the true digit. The 1s form the lobe on one side and the 0s lie on the opposite side, consistent with the pattern of $\mathbf{v}_1$ in Figure 7. The other digits overlap.

2.5 Loadings of gene expression

For the tumour data the principal directions live in the space of genes, and the loadings acquire a biological meaning. The score of patient i along $\mathbf{v}_l$ is $w_i^{(l)} = \sum_g v_{lg}(x_{ig} - \bar{x}_g)$, a weighted sum of the deviations of all gene expressions from their means. Genes with large loadings of the same sign rise and fall together across the patients, and genes with large loadings of opposite signs vary in opposite directions. A principal direction whose large loadings fall on a group of genes therefore identifies co-regulated genes, a candidate expression program, and the score of a patient measures how strongly that program is active in the patient's tumour. Figure 9 shows the loadings of the first principal direction of the tumour data, computed from the 2000 genes of largest variance, each standardised to unit variance, over all 801 samples. The first component carries 20.3% of the variance. Its loadings are spread over many genes. The 40

largest of them carry 8.1% of the squared loadings, four times the share of 40 equal loadings, so the component is a broad program of co-varying genes and no small set of genes defines it. The scores show what the program separates. The mean score of the kidney carcinomas (KIRC) is -39.8, those of the breast (BRCA) and prostate (PRAD) carcinomas are 12.5 and 15.6, and the lung and colon types lie between. The first component thus contrasts the kidney tumours with the breast and prostate tumours, a distinction found without the tumour types.

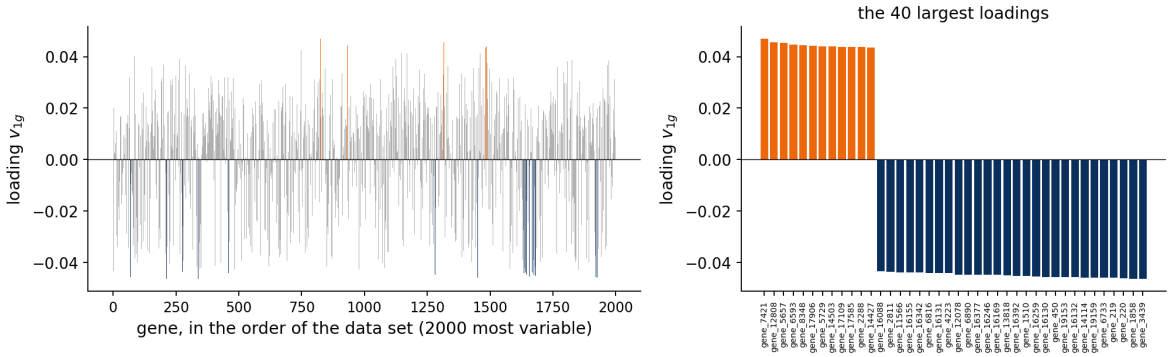


Figure 9: Loadings v_{1g} of the first principal component of the tumour data (2000 most variable genes, standardised, 801 samples). Left, all 2000 loadings in the order of the genes in the data set, with the 40 largest in absolute value coloured (orange positive, navy negative). Right, these 40 loadings sorted, labelled by the anonymised gene names.

The first is the number of samples. With $n = 801$ patients and $D = 20\,531$ genes, the centred data matrix has rank at most $n-1 = 800$, because the centred rows sum to zero. At most 800 eigenvalues of the $20\,531 \times 20\,531$ covariance matrix are positive, and the remaining eigenvalues vanish. Forming C is wasteful in this case, and the SVD supplies the principal directions from the much smaller $n \times n$ matrix $X_c X_c^\top$. If $X_c X_c^\top \mathbf{u}_l = s_l^2 \mathbf{u}_l$, then multiplying by $X_c^\top$ gives

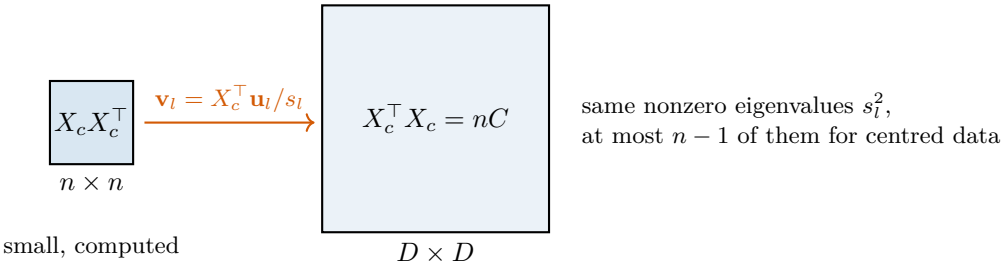


Figure 10: For $D \gg n$, as for the tumour data, the principal directions are obtained from the small $n \times n$ matrix $X_c X_c^\top$. Its eigenvectors $\mathbf{u}_l$ give the principal directions $\mathbf{v}_l$ by equation (2.21), and both matrices have the same nonzero eigenvalues.

$$X_c^\top X_c \, (X_c^\top \mathbf{u}_l) = s_l^2 \, (X_c^\top \mathbf{u}_l), \qquad \mathbf{v}_l = \frac{X_c^\top \mathbf{u}_l}{s_l}, \tag{2.21}$$

where the division by s_l normalises the vector, since $\|X_c^\top \mathbf{u}_l\|^2 = \mathbf{u}_l^\top X_c X_c^\top \mathbf{u}_l = s_l^2$. Each eigenvector of the small matrix thus yields a principal direction in gene space (Figure 10). The equation also shows that every principal direction is a linear combination of the observed expression profiles. With few patients the directions are fitted to those patients, and their loadings on individual genes are uncertain. Loadings that would carry a biological interpretation must be checked for stability, for example by recomputing them on random halves of the patients.

The second feature is the scale of the genes. The covariance matrix weights each gene by its variance, so a few genes with very large variance dominate the first components. Standardising each gene to unit variance, that is working with the correlation matrix, gives every gene the same weight, at the price of amplifying genes whose small variations are mostly measurement noise. A common compromise, which we adopt for the anomaly detection in Section 7.8, keeps the 2000 genes of largest variance and standardises them. The selection by variance removes the many genes whose expression hardly changes between patients and which standardisation would otherwise raise to unit variance with mostly noise, and the standardisation then gives each remaining gene the same weight. In the data used here the genes are anonymised, so the loadings cannot be mapped to named genes or pathways. With named genes the standard procedure lists the genes of largest absolute loading on a component and tests whether known pathways are overrepresented among them.

2.6 Transactions that are already principal components

The transaction data illustrate the opposite case. Their columns $V_1, \ldots, V_{28}$ are themselves principal components of the confidential original variables. Principal components are uncorrelated, so after each column is standardised the covariance matrix of these columns is close to the identity matrix. For the identity every direction carries the same variance, the spectrum is flat, and no plane of small dimension is preferred. For 29 perfectly uncorrelated standardised columns two components would retain $2/29 \approx 6.9\%$ of the variance. We standardise with the means and standard deviations of the normal training transactions and fit PCA to them. The first two components retain 9.9% of the variance, little more than the flat value, so the transactions have almost no linear low-dimensional structure. Figure 43, discussed in Section 7.7, shows the projection. Some frauds lie at the edge of the normal cloud and many lie inside it. A two-dimensional linear picture is not the tool that will find them.

2.7 What a linear projection cannot do

PCA assumes that the structure of interest is a flat subspace. When the data lie near a curved set, a projection onto a plane superposes points that are far apart along the set. Figure 11 shows the simplest case, points on a curve in the plane. The best line through them is the first principal direction, and projecting onto it maps the two ends of the curve onto the same interval. Points that are distant along the curve receive the same coordinate, and the one-dimensional structure of the data, which a single number along the curve would describe without loss, is destroyed.

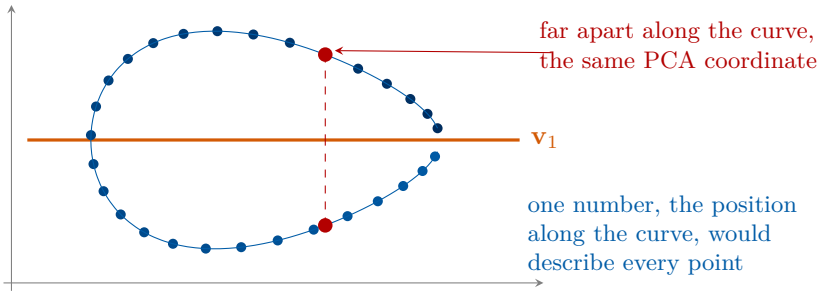


Figure 11: A limitation of PCA. Points on a closed curve (colour changes along the curve) are projected onto the first principal direction. Points from opposite sides of the curve (red) receive nearly the same coordinate, although they are far apart along the curve. A nonlinear map that unrolls the curve would preserve its one-dimensional structure.

The MNIST map of Figure 8 shows the same failure in 784 dimensions. The digits form groups that a classifier separates easily, as earlier lectures showed, yet the best plane through the data superposes them. The groups are arranged around curved, possibly disconnected sets, and a single flat projection cannot separate them. Two kinds of nonlinear methods address this limitation. The first gives up the global linear map and asks only that neighbouring points remain neighbours in a low-dimensional picture, which is the idea of t-SNE in the next section. The second keeps a map from data to a few coordinates and back, but lets it be nonlinear, which leads to autoencoders in Section 5.

3 Neighbourhood-preserving maps

3.1 Preserving neighbours instead of variance

The curve of Figure 11 would be described correctly by any map that keeps neighbouring points next to each other, even if it distorts large distances. Such a map unrolls the curve, because points adjacent along the curve stay adjacent and nothing else is required. This observation suggests a different criterion for a low-dimensional picture. Instead of the variance along a fixed plane, the picture should preserve, for every sample, the identity of its nearest neighbours. Large distances are given up deliberately. They are, by equation (1.3), the least informative distances in high dimension, and a map of a curved set cannot preserve them anyway.

t-distributed stochastic neighbour embedding (t-SNE) [11], which grew out of the earlier stochastic neighbour embedding (SNE) [12], turns this idea into an optimisation problem. Every sample $\mathbf{x}_i$ is represented by a point $\mathbf{y}_i \in \mathbb{R}^d$ of a map, usually with $d = 2$. The neighbourhood relations are described by probabilities, once among the data and once among the map points, and the map points are moved until the two descriptions agree. The nd map coordinates are themselves the unknowns. Unlike PCA, t-SNE produces no function $\mathbf{y} = f(\mathbf{x})$ that could be applied to a new sample.

3.2 Similarities in the data

Around each sample $\mathbf{x}_i$ we place a Gaussian of width σ_i and define the probability that $\mathbf{x}_i$ would choose $\mathbf{x}_j$ as its neighbour if neighbours were chosen in proportion to the Gaussian density,

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{l \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_l\|^2/2\sigma_i^2)}, \qquad p_{i|i} = 0. \tag{3.1}$$

For fixed i the numbers $p_{j|i}$ form a probability distribution over the $n-1$ other samples. The Gaussian makes the probability fall quickly with distance, so that only samples within a few widths σ_i of $\mathbf{x}_i$ receive appreciable probability.

The width cannot be the same for all samples. In a dense region a small width already includes many neighbours, while in a sparse region the same width would include none. t-SNE therefore fixes the effective number of neighbours of every sample and adjusts σ_i to it. The effective number is measured by the perplexity,

$$\mathrm{Perp}_i = 2^{H_i}, \qquad H_i = -\sum_{j \neq i} p_{j|i} \log_2 p_{j|i}, \tag{3.2}$$

where H_i is the Shannon entropy of the neighbour distribution in bits. If $\mathbf{x}_i$ had k equally

likely neighbours and no others, then $p_{j|i} = 1/k$ for those k samples and

$$H_i = -k \cdot \frac{1}{k} \log_2 \frac{1}{k} = \log_2 k, \qquad \mathrm{Perp}_i = 2^{\log_2 k} = k, \tag{3.3}$$

so the perplexity counts neighbours. For a Gaussian distribution of probabilities it is a smooth count of the samples within roughly one width. A larger σ_i spreads the probability over more samples and raises the entropy, and H_i increases monotonically with σ_i. The width that gives the prescribed perplexity, the same for all i, is therefore found for each sample separately by bisection, which repeatedly halves an interval of widths known to contain the solution until the perplexity matches. The perplexity is the main parameter of the method, and values between 5 and 100 are common, with 30 the default of standard implementations. Figure 12 shows why the width must depend on the sample.

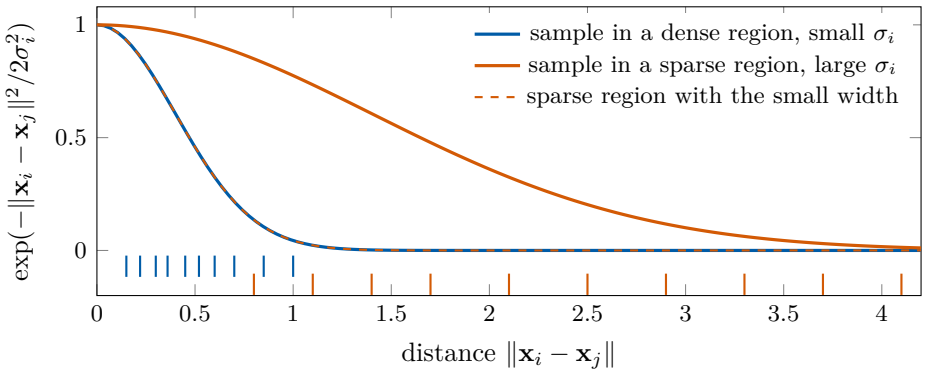


Figure 12: Neighbour weights (3.1) before normalisation, for a sample in a dense region (blue) and one in a sparse region (orange). The ticks at the bottom mark the distances of the ten nearest neighbours of each. With the width that suits the dense region, the sparse sample would give almost no weight to any neighbour (dashed). Choosing σ_i for a fixed perplexity gives both samples about the same effective number of neighbours. Schematic distances.

The conditional probabilities are not symmetric, since $\sigma_i \neq \sigma_j$ in general. t-SNE uses the symmetric joint probabilities

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \qquad \sum_{i \neq j} p_{ij} = \frac{1}{2n}\Big(\sum_i \sum_j p_{j|i} + \sum_j \sum_i p_{i|j}\Big) = \frac{n+n}{2n} = 1, \tag{3.4}$$

which form one probability distribution P over all pairs of samples. Every sample takes part in it with a total weight $\sum_j p_{ij} \geq \sum_j p_{j|i}/2n = 1/2n$, so even a sample far from all others keeps a place in the map.

3.3 Similarities in the map and the crowding problem

In the map, the similarity of two points is measured by an analogous distribution Q over pairs. The original SNE used a Gaussian kernel in the map as well. This choice fails for a geometric reason that is again a consequence of high dimension, the crowding problem. In D dimensions up to $D + 1$ points can lie at equal mutual distances, as the vertices of a regular simplex, while in the plane at most three can (Figure 13). More generally the volume of a ball of radius r grows like r^D, so a sample in a uniformly filled region of dimension D has 2^D times more samples within distance r than within $r/2$. In a two-dimensional map the ratio is only 4. When the neighbourhoods of high dimension are flattened into the plane, the samples at moderate distances from $\mathbf{x}_i$ cannot all be placed at moderate distances from $\mathbf{y}_i$. There is

not enough room, and many of them must be placed much farther away than their distance in the data suggests.



plane, $d = 2$: at most 3 points

space, $D = 3$: 4 points

points at equal mutual distances: $D + 1$ in $\mathbb{R}^D$, hence 785 in the MNIST pixel space and 3 in a 2D map

Figure 13: The crowding problem. Equal mutual distances can be realised by $D + 1$ points in D dimensions, the vertices of a regular simplex, but only by three in the plane. Groups that are roughly equidistant in high dimension cannot all be equidistant in a two-dimensional map, and some pairs at moderate distance must be placed far apart.

With a Gaussian in the map such a pair, placed far away, receives a similarity q_{ij} that is essentially zero, while its p_{ij} is small but not zero. The objective introduced in the next subsection penalises this mismatch and pulls such pairs back. The result is a map in which all groups are squeezed together in the centre. t-SNE removes the difficulty by using a kernel with a heavy tail in the map, the Student-t distribution with one degree of freedom,

$$q_{ij} = \frac{\left(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2\right)^{-1}}{\sum_{l \neq m} \left(1 + \|\mathbf{y}_l - \mathbf{y}_m\|^2\right)^{-1}}, \qquad q_{ii} = 0. \tag{3.5}$$

The denominator sums over all pairs of map points, so Q is a probability distribution over pairs, of the same kind as P. The map kernel has no width. The map coordinates are free parameters, so a width would only rescale the whole map and is absorbed into the positions. One degree of freedom gives the heaviest tail among the Student-t kernels, a simple closed form and a cheap gradient, equation (3.11), and it is the choice of the original method [11].

Both kernels equal one at $r = 0$ and decrease smoothly at small distances, but they differ completely at large ones. The Gaussian decays like $e^{-r^2/2}$ and the Student-t only like $1/r^2$. A given small similarity is therefore reached at a much larger distance. Setting each kernel equal to 0.01 gives

$$e^{-r_{\mathrm{G}}^2/2} = 0.01 \;\Rightarrow\; r_{\mathrm{G}} = \sqrt{2 \ln 100} = 3.03, \qquad \frac{1}{1 + r_{\mathrm{t}}^2} = 0.01 \;\Rightarrow\; r_{\mathrm{t}} = \sqrt{99} = 9.95, \tag{3.6}$$

a distance larger by a factor of 3.3. A pair with a small but nonzero p_{ij}, a moderately distant pair in the data, can be placed far apart in the map and still receive a matching q_{ij}. The heavy tail provides the room that the plane lacks, and as a consequence gaps open between groups in the map. Figure 14 compares the two kernels, and Figure 15 shows the resulting transfer of neighbourhoods from the data to the map.

3.4 The objective and its gradient

The map is found by making Q as close as possible to P. The measure of closeness is the Kullback–Leibler (KL) divergence,

$$\mathcal{C}(\mathbf{y}_1, \ldots, \mathbf{y}_n) = \mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}, \tag{3.7}$$

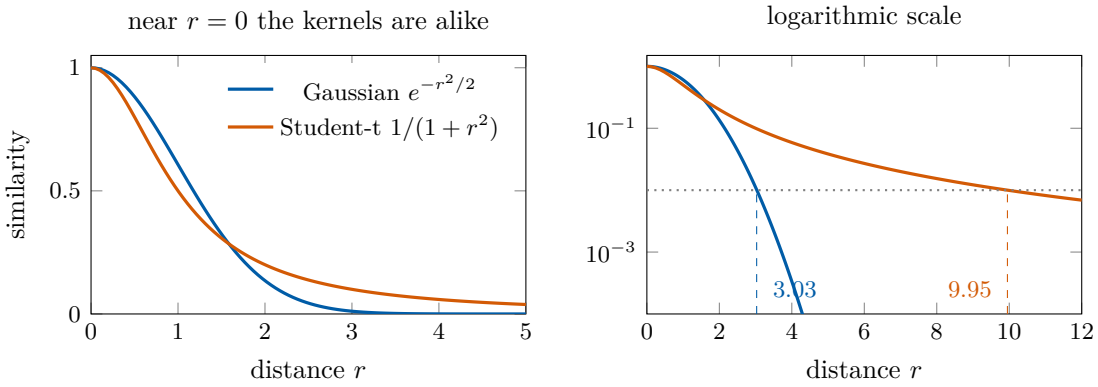


Figure 14: The Gaussian kernel, used for the data and in the original SNE map, and the Student-t kernel of the t-SNE map. Left, at small distances they are similar. Right, on a logarithmic scale, a similarity of 0.01 is reached at $r = 3.03$ by the Gaussian and at $r = 9.95$ by the Student-t, equation (3.6). The heavy tail lets moderately distant pairs sit far apart in the map.

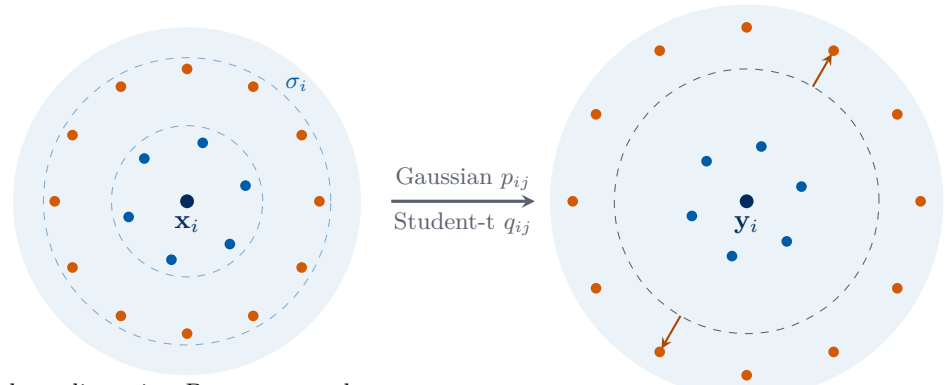


Figure 15: Transfer of a neighbourhood from the data to the map. Left, in high dimension the sample $\mathbf{x}_i$ has a few close neighbours (blue) and many neighbours at moderate distance (orange), for which the Gaussian of width σ_i gives small but nonzero p_{ij}. Right, in the plane there is no room for all of them at the corresponding distance (grey circle). The Student-t kernel allows them to be placed farther out with a matching q_{ij}, which separates groups in the map.

which is nonnegative and vanishes only for $Q = P$. Nonnegativity follows from Jensen's inequality for the concave logarithm,

$$-\operatorname{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{q_{ij}}{p_{ij}} \leq \log \sum_{i \neq j} p_{ij} \frac{q_{ij}}{p_{ij}} = \log \sum_{i \neq j} q_{ij} = 0, \tag{3.8}$$

with equality only when q_{ij}/p_{ij} is the same for all pairs. Since both distributions sum to one, this constant is one, so that $Q = P$. The divergence is asymmetric, and this asymmetry decides what the map preserves. A pair of neighbours in the data that is placed far apart in the map has a large p_{ij} and a small q_{ij} and contributes a large positive term $p_{ij} \log(p_{ij}/q_{ij})$. A pair of distant samples placed close together has a small p_{ij}, and its term is small whatever q_{ij} is. The objective therefore enforces that neighbours stay neighbours, and it barely constrains whether non-neighbours stay apart. t-SNE preserves the local structure of the data and gives up the global one.

The map points are found by gradient descent on $\mathcal{C}$. Only q_{ij} depends on the map. Write $E_{ij} = (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$ and $Z = \sum_{l \neq m} E_{lm}$, so that $q_{ij} = E_{ij}/Z$. Since $\sum_{i \neq j} p_{ij} = 1$,

$$\mathcal{C} = \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log E_{ij} + \log Z. \tag{3.9}$$

The point $\mathbf{y}_i$ enters the pairs (i, j) and (j, i), which are equal, and the derivative of one kernel is

$$\frac{\partial E_{ij}}{\partial \mathbf{y}_i} = -E_{ij}^2 \, 2(\mathbf{y}_i - \mathbf{y}_j). \tag{3.10}$$

The second term of equation (3.9) contributes $-2 \sum_j p_{ij} E_{ij}^{-1} \partial E_{ij}/\partial \mathbf{y}_i = 4 \sum_j p_{ij} E_{ij} (\mathbf{y}_i - \mathbf{y}_j)$, and the third contributes $Z^{-1} \, 2 \sum_j \partial E_{ij}/\partial \mathbf{y}_i = -4 \sum_j q_{ij} E_{ij} (\mathbf{y}_i - \mathbf{y}_j)$. Together,

$$\frac{\partial \mathcal{C}}{\partial \mathbf{y}_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) \, \frac{\mathbf{y}_i - \mathbf{y}_j}{1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2}. \tag{3.11}$$

The gradient has the form of a sum of pairwise forces. A step of gradient descent moves $\mathbf{y}_i$ along $-\partial \mathcal{C}/\partial \mathbf{y}_i$, that is towards $\mathbf{y}_j$ when $p_{ij} > q_{ij}$ and away from it when $p_{ij} < q_{ij}$. Pairs that are closer in the data than in the map attract, and pairs that are closer in the map than in the data repel, like springs whose rest length is set by the data (Figure 16). The factor $(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$ weakens the force between distant map points, so that well separated groups exert little force on each other and can drift apart.

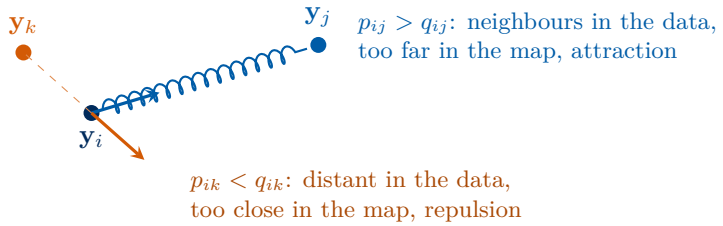


Figure 16: The gradient (3.11) as pairwise forces on the map point $\mathbf{y}_i$. A pair with $p_{ij} > q_{ij}$ is pulled together, a pair with $p_{ik} < q_{ik}$ is pushed apart. The map is in equilibrium when the forces on every point balance.

The objective is not convex, and its minimum depends on the starting map. Common practice starts the map points from the first two principal components of the data, which fixes the large-scale arrangement in a reproducible way, and multiplies all p_{ij} by a constant factor

during the first iterations. Since the q_{ij} still sum to one, this strengthens the attraction between neighbours relative to the repulsion, so that tightly connected groups contract early and can move past each other before their mutual positions are settled. The sums over all pairs make one iteration cost of order n^2 operations, and standard implementations approximate the repulsive forces of distant groups, as in the Barnes–Hut method of gravitational N-body simulations. With a few thousand samples a map takes seconds to minutes on a single processor. It is also common to run t-SNE on the first few tens of principal components of the data, which removes the noise carried by the last components and reduces the cost of the distances. For MNIST we use the 50 components that retain 82.5% of the variance (Table 2).

3.5 Maps of digits

A small synthetic example shows what t-SNE achieves where PCA fails. Eight groups of 60 points are placed at the corners of a cube in three dimensions. Each group is elongated and randomly oriented, so that the groups pass through each other and neither a view of the cloud nor its PCA projection shows how many there are (Figure 17). The t-SNE map with perplexity 30 shows eight separate groups, and coloured back in three dimensions they are the groups that generated the data. To quantify the separation we run the k-means clustering of Section 4.1 with eight clusters on each two-dimensional picture and compare the result with the true groups by the adjusted Rand index of Section 4.2, which is 1 for perfect agreement and close to 0 for agreement by chance. The index is 0.46 on the PCA projection and 0.96 on the t-SNE map.

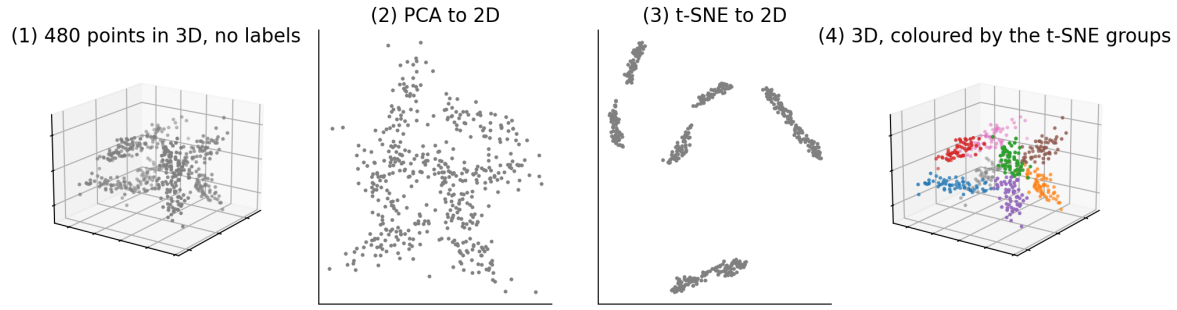


Figure 17: Eight elongated groups of 60 points in three dimensions. (1) The cloud without labels. (2) Its projection onto the first two principal components. (3) The t-SNE map with perplexity 30, in which eight groups appear. (4) The three-dimensional points coloured by the groups that k-means finds on the t-SNE map.

On the 5000 MNIST test digits, represented by their 50 principal components, t-SNE with perplexity 30 produces Figure 18. Without colours the map shows about ten separate islands, and the colours show that they correspond to the digits. The 4s, 7s and 9s, which differ by small strokes, form adjacent islands that touch. The map contains no information about the digits beyond the pixel values, so the separation shows that each digit forms a connected set of neighbours in pixel space, although the sets are not separated by any plane.

The perplexity sets the scale of the neighbourhoods. Figure 19 shows maps of the first 2000 of the same digits at perplexities 5, 30 and 100. At perplexity 5 only the closest neighbours enter, and the digits form loose groups broken into strands. At larger perplexity the map becomes smoother, and at 100 most digits form compact islands. The 4s, 7s and 9s remain adjacent at every setting, which indicates that the proximity is in the data.

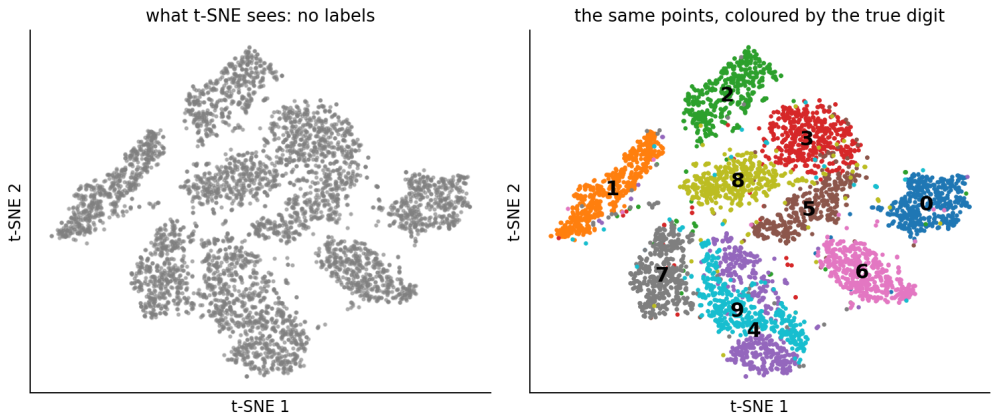


Figure 18: t-SNE map of the 5000 MNIST test digits (perplexity 30, computed from 50 principal components). Left, the map without labels. Right, the same points coloured by the true digit, with each digit written at the median of its points.

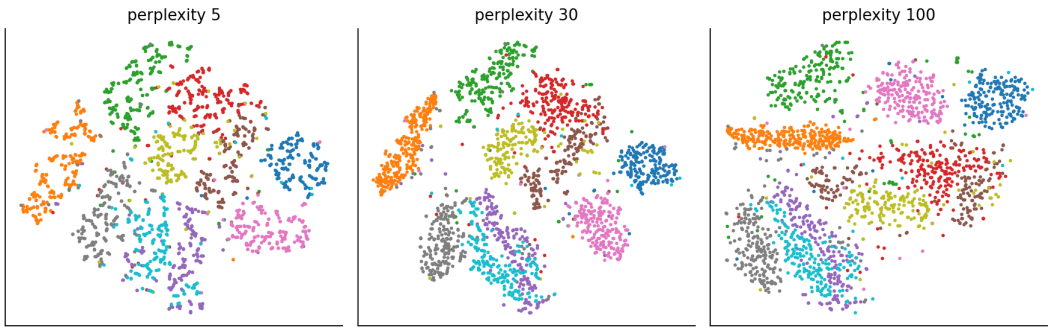


Figure 19: t-SNE maps of 2000 MNIST test digits at perplexity 5, 30 and 100, coloured by the true digit. Small perplexity keeps only very close neighbours, large perplexity smooths the map.

3.6 What a t-SNE map shows

A t-SNE map is easily over-interpreted, and the objective (3.7) determines which of its features carry meaning [13]. The objective depends only on distances between map points, so a rotated, reflected or translated map is equally good, and the position of a group in the map and the direction of the axes carry no meaning. The separation between groups carries little meaning, because pairs of distant samples have almost vanishing p_{ij} and hardly enter the objective, while the heavy tail of the kernel pushes groups apart. A wide gap does not indicate very different data. The size of a group carries little meaning either, since the widths σ_i adapt to the local density of samples, so that a dense group and a sparse group of the same number of samples come out with similar extent. Even distances inside a group are not to scale beyond the nearest neighbours, roughly as many as the perplexity.

What the map shows reliably is which samples are close neighbours. A group that appears consistently over several random starts and perplexities indicates a set of samples that are mutual neighbours in the data. Since the map points are free parameters and not values of a function, a new sample has no map point until the whole map is recomputed with it, and the map cannot be used to encode new data or to reconstruct a sample from map coordinates.

Table 3: PCA and t-SNE compared.

	PCA	t-SNE
criterion	largest variance, smallest reconstruction error	neighbour probabilities Q close to P
map	linear function $\mathbf{w} = V_d^\top(\mathbf{x} - \bar{\mathbf{x}})$	free map points $\mathbf{y}_i$, no function
new sample	projected with the same V_d	requires a new map
reconstruction	$\hat{\mathbf{x}} = \bar{\mathbf{x}} + V_d\mathbf{w}$	none
preserves	global variance, large distances	close neighbours
fails for	curved or disconnected structure	distances between and sizes of groups

Maps made by PCA and t-SNE show groups to the eye. A scientist who wants to count the groups, to assign each sample to one of them, or to check whether the groups agree with a known classification needs an explicit procedure that partitions the samples, and clustering supplies such a procedure.

4 Clustering

4.1 k-means as minimal within-cluster variance

A partition of the samples into k groups is good, in the simplest sense, if the samples of each group lie close to a common centre. The k-means method makes this precise [14]. It seeks disjoint sets of sample indices $S_1, \ldots, S_k$, whose union contains every sample once, and centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k \in \mathbb{R}^D$ that minimise the total squared distance of the samples to the centres of their clusters,

$$J(\{S_c\}, \{\boldsymbol{\mu}_c\}) = \sum_{c=1}^{k} \sum_{i \in S_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|^2. \tag{4.1}$$

The number of clusters k is chosen in advance. The objective contains two kinds of unknowns, a discrete assignment of samples to clusters and continuous centres, and it is minimised by alternating between them.

With the centres fixed, each term of equation (4.1) depends on the cluster of one sample only, and it is smallest when the sample joins its nearest centre,

$$S_c = \{\, i : \ \|\mathbf{x}_i - \boldsymbol{\mu}_c\| \le \|\mathbf{x}_i - \boldsymbol{\mu}_{c'}\| \text{ for all } c' \,\} \qquad \text{(assignment step)}, \tag{4.2}$$

with ties broken arbitrarily. With the assignment fixed, the objective is a sum of separate quadratic functions of the centres. Setting the derivative with respect to $\boldsymbol{\mu}_c$ to zero,

$$\frac{\partial J}{\partial \boldsymbol{\mu}_c} = -2 \sum_{i \in S_c} (\mathbf{x}_i - \boldsymbol{\mu}_c) = 0 \quad \Longrightarrow \quad \boldsymbol{\mu}_c = \frac{1}{|S_c|} \sum_{i \in S_c} \mathbf{x}_i \qquad \text{(update step)}, \tag{4.3}$$

places each centre at the mean of its samples, which explains the name of the method. The alternation of the two steps is Lloyd's algorithm. Neither step can increase J, because each minimises it over one set of unknowns with the other held fixed. If ties in the assignment step are broken in favour of the current cluster, the assignment changes only when the objective strictly decreases, and otherwise J stays the same. There are only finitely many partitions of n samples, so no partition is visited twice and the algorithm stops after finitely many iterations, when no sample changes its cluster. The assignment step divides space into the Voronoi cells of the centres, the regions of points nearest to each centre, which are convex regions bounded by the perpendicular bisector planes of pairs of centres (Figure 20).

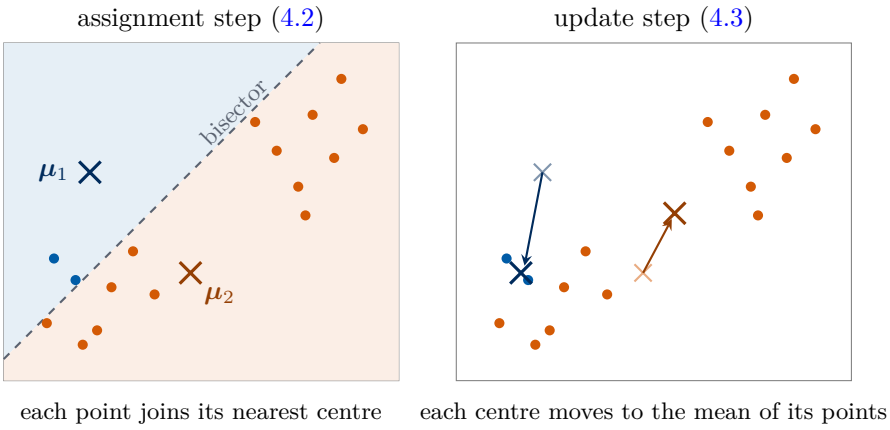


Figure 20: One iteration of Lloyd's algorithm with two clusters. Left, the assignment step. The dashed perpendicular bisector of the two centres (crosses) divides the plane into two Voronoi cells, and each point takes the colour of the centre whose cell contains it. Right, the update step. Each centre moves to the mean of the points assigned to it. In this example six points of the lower group are still assigned to the orange centre, which therefore moves only part of the way to the upper group. The next assignment step, with the bisector of the moved centres, assigns all points of the lower group to the blue centre.

The objective has a second interpretation. Writing $\mathbf{x}_i - \bar{\mathbf{x}} = (\mathbf{x}_i - \boldsymbol{\mu}_c) + (\boldsymbol{\mu}_c - \bar{\mathbf{x}})$ for $i \in S_c$ and using that the deviations $\mathbf{x}_i - \boldsymbol{\mu}_c$ sum to zero within each cluster when $\boldsymbol{\mu}_c$ is the cluster mean, the total sum of squares splits as

$$\sum_{i=1}^{n} \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 = \underbrace{\sum_{c=1}^{k} \sum_{i \in S_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|^2}_{J,\ \text{within clusters}} + \underbrace{\sum_{c=1}^{k} |S_c| \, \|\boldsymbol{\mu}_c - \bar{\mathbf{x}}\|^2}_{\text{between clusters}} . \tag{4.4}$$

The left side is fixed by the data, so minimising the within-cluster sum of squares is the same as maximising the spread of the cluster means. k-means looks for compact clusters with

well separated centres, and since it measures everything by Euclidean distance to a centre it favours clusters that are roughly round and of similar extent.

Figure 21 follows the algorithm on 500 points drawn around five centres in the plane, with five starting centres deliberately placed badly, at the five points nearest the lower left corner. The assignment and the centres change rapidly in the first iterations and slowly later, and the objective printed above each panel falls quickly and then slowly. After 11 iterations no point changes its cluster. The final clusters agree with the five groups that generated the points with an adjusted Rand index (Section 4.2) of 0.93. A library implementation of Lloyd's algorithm, started from the same five centres, produces the same labels for all 500 points and the same objective to floating-point rounding.

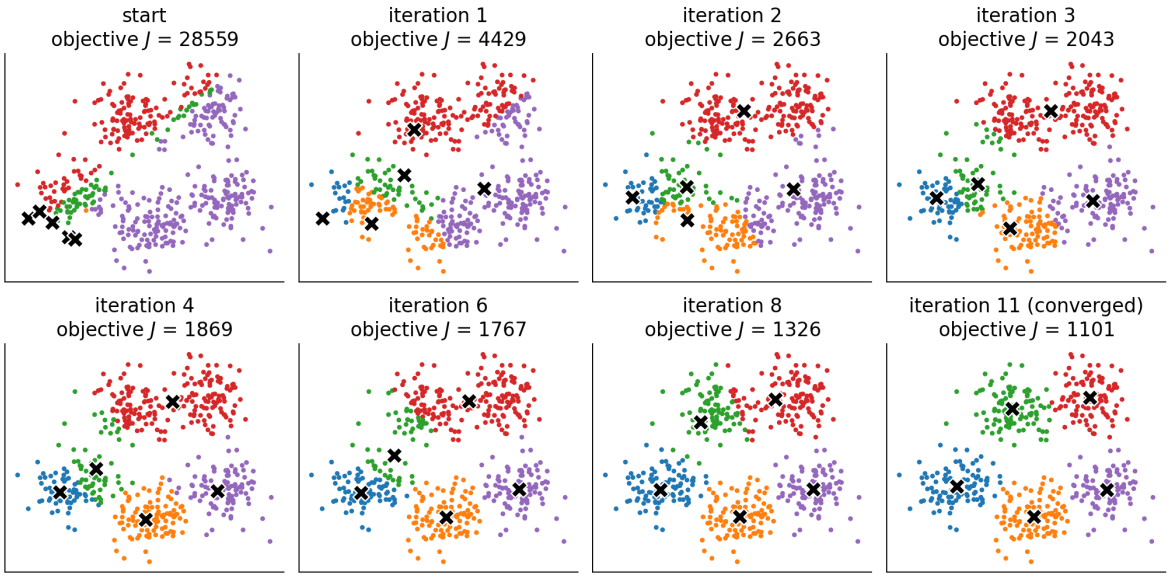


Figure 21: Lloyd's algorithm on 500 points in the plane with $k = 5$. The colours show the assignment and the crosses the centres at the start, after iterations 1, 2, 3, 4, 6 and 8, and at convergence after iteration 11. The objective J of equation (4.1) is printed above each panel.

Lloyd's algorithm finds a local minimum of J, and different starting centres can lead to different local minima. A start with two centres inside one true group and none in another can converge to a partition that splits one group and merges two others, and no sequence of assignment and update steps undoes the error, since each step only decreases J. The k-means++ initialisation [15] reduces this risk. It chooses the first centre at random among the samples and every further centre among the samples with probability proportional to the squared distance from the nearest centre already chosen, which spreads the starting centres over the data. In addition, the algorithm is run from several such starts, and the partition with the smallest J is kept. All clusterings reported in the rest of the chapter use ten starts.

4.2 Comparing a clustering with known labels

When labels exist for the samples, as for the digits or the blood cells, they allow us to check whether the clusters found without them correspond to a known distinction. A cluster index and a label are names of groups, and the names of clusters are arbitrary, so cluster 3 may correspond to digit 7. The comparison must therefore be invariant under renaming. The basic object is the contingency table N_{gc}, the number of samples with label g in cluster c. A perfect clustering has one nonzero entry in each row and each column, and the numbering of

the columns is irrelevant.

The Rand index [16] summarises the table by counting pairs of samples. Among the $N_2 = \binom{n}{2} = n(n-1)/2$ pairs, a pair agrees if both partitions put its two samples in the same group or both put them in different groups. The Rand index RI is the fraction of agreeing pairs. It is computed from the table as follows. The number of pairs together in both partitions, the number of pairs together according to the labels and the number together according to the clusters are

$$T_\mathrm{b} = \sum_{g,c} \binom{N_{gc}}{2}, \qquad T_\mathrm{l} = \sum_{g} \binom{N_{g\cdot}}{2}, \qquad T_\mathrm{c} = \sum_{c} \binom{N_{\cdot c}}{2}, \tag{4.5}$$

where $N_{g\cdot} = \sum_c N_{gc}$ and $N_{\cdot c} = \sum_g N_{gc}$ are the row and column sums. The pairs apart in both partitions number $N_2 - T_\mathrm{l} - T_\mathrm{c} + T_\mathrm{b}$, since $T_\mathrm{l} + T_\mathrm{c} - T_\mathrm{b}$ pairs are together in at least one of them, and therefore

$$\mathrm{RI} = \frac{T_\mathrm{b} + (N_2 - T_\mathrm{l} - T_\mathrm{c} + T_\mathrm{b})}{N_2} = \frac{N_2 + 2T_\mathrm{b} - T_\mathrm{l} - T_\mathrm{c}}{N_2}. \tag{4.6}$$

The Rand index is high even for random clusters, because most pairs of samples are apart in both partitions when there are many groups. It is corrected for chance by comparing it with its mean under random assignment. If the samples are redistributed at random among clusters of the given sizes, which amounts to a random permutation of the samples over the n cluster positions, a fixed pair of samples lands in cluster c with probability $N_{\cdot c}(N_{\cdot c}-1)/n(n-1)$, the fraction of ordered pairs of positions that both belong to cluster c. The probability that the pair shares a cluster is therefore

$$\sum_c \frac{N_{\cdot c}(N_{\cdot c}-1)}{n(n-1)} = \frac{\sum_c \binom{N_{\cdot c}}{2}}{\binom{n}{2}} = \frac{T_\mathrm{c}}{N_2}. \tag{4.7}$$

Each of the T_l pairs that the labels put together falls into a common cluster with this probability, so the expected number of pairs together in both partitions is $\mathbb{E}[T_\mathrm{b}] = T_\mathrm{l}T_\mathrm{c}/N_2$. The adjusted Rand index [17] subtracts this expectation and divides by the largest attainable excess,

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{1 - \mathbb{E}[\mathrm{RI}]} = \frac{T_\mathrm{b} - T_\mathrm{l}T_\mathrm{c}/N_2}{\frac{1}{2}(T_\mathrm{l} + T_\mathrm{c}) - T_\mathrm{l}T_\mathrm{c}/N_2}, \tag{4.8}$$

where the second form follows from the first because RI is linear in T_b, by equation (4.6), so that $\mathrm{RI} - \mathbb{E}[\mathrm{RI}] = 2(T_\mathrm{b} - \mathbb{E}[T_\mathrm{b}])/N_2$ and $1 - \mathbb{E}[\mathrm{RI}] = (T_\mathrm{l} + T_\mathrm{c} - 2\,\mathbb{E}[T_\mathrm{b}])/N_2$. The adjusted index equals 1 for identical partitions, is close to 0 for a random clustering, and can be negative.

Table 4: Contingency table N_{gc} of six samples with the labels $\{1,2,3\}$ and $\{4,5,6\}$ and the clusters $\{1,2\}$, $\{3,4,5\}$ and $\{6\}$, with the row sums $N_{g\cdot}$ and the column sums $N_{\cdot c}$.

	cluster $\{1,2\}$	cluster $\{3,4,5\}$	cluster $\{6\}$	$N_{g\cdot}$
label $\{1,2,3\}$	2	1	0	3
label $\{4,5,6\}$	0	2	1	3
$N_{\cdot c}$	2	3	1	6

Six samples illustrate the two indices (Figure 22 and Table 4). The labels split them into $\{1,2,3\}$ and $\{4,5,6\}$, and the clusters into $\{1,2\}$, $\{3,4,5\}$ and $\{6\}$. Of the 15 pairs, two are together in both partitions, $\{1,2\}$ and $\{4,5\}$, and seven are apart in both, so the Rand index is 0.60. Clusters of the same sizes drawn at random would agree on most of these pairs as

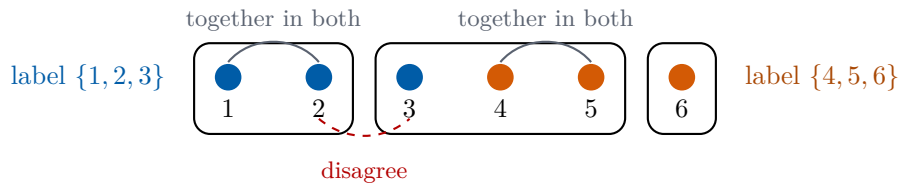


Figure 22: Six samples coloured by their label and enclosed by their cluster. A pair agrees when both partitions put it together, such as $\{1, 2\}$ and $\{4, 5\}$, or both put it apart, such as $\{1, 6\}$. The pair $\{2, 3\}$ shares a label but not a cluster and disagrees. Nine of the 15 pairs agree.

well, and after the correction (4.8) the adjusted index is only 0.12. The clustering captures the labels weakly.

Table 5 shows the two indices on 600 points drawn around four centres in the plane (Figure 23). k-means with four clusters recovers the groups almost perfectly. With two clusters, each containing two groups, the Rand index is still 0.750. Random labels reach a Rand index of 0.626, while their adjusted index is zero to three decimals, which shows why the adjusted index is the one to report.

Table 5: Rand index and adjusted Rand index between the four true groups of 600 points and three partitions of them (Figure 23).

partition	RI	ARI
k-means, $k = 4$	0.997	0.991
k-means, $k = 2$	0.750	0.499
random labels	0.626	0.000

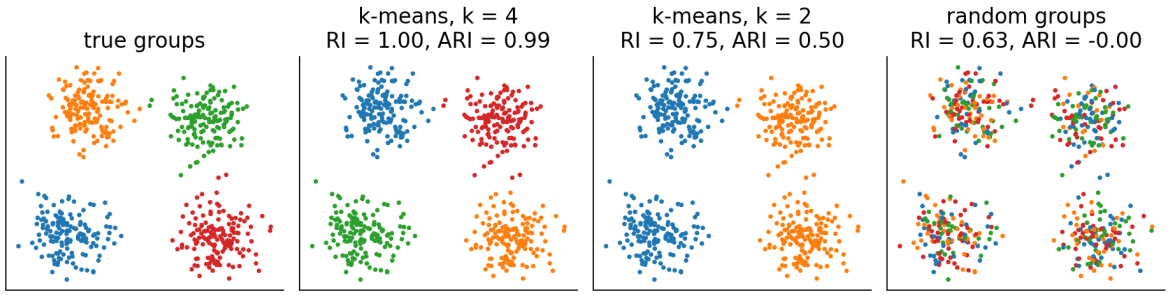


Figure 23: 600 points around four centres. From left to right, the true groups, k-means with four clusters, k-means with two clusters and random labels, with the Rand index and the adjusted Rand index of each partition against the true groups.

4.3 Choosing the number of clusters

k-means requires k, and without labels it has to be chosen from the data. The objective itself cannot decide, because J decreases whenever k grows, more centres giving shorter distances, down to $J = 0$ for $k = n$. The elbow heuristic plots the minimal J against k and looks for the value after which it decreases only slowly. Real data often give a smooth curve without a clear elbow, and a second criterion is needed.

The silhouette [18] asks for each sample whether it lies well inside its own cluster and far from

the next one. Let $\bar{\delta}_i^{\text{in}}$ be the mean distance of sample i to the other samples of its cluster and $\bar{\delta}_i^{\text{out}}$ its mean distance to the samples of the nearest other cluster. The silhouette of sample i is

$$\text{sil}_i = \frac{\bar{\delta}_i^{\text{out}} - \bar{\delta}_i^{\text{in}}}{\max\left(\bar{\delta}_i^{\text{in}}, \bar{\delta}_i^{\text{out}}\right)} \in [-1, 1]. \tag{4.9}$$

It is close to 1 when the sample is much closer to its own cluster than to any other, close to 0 when it lies between two clusters, and negative when it would fit better into the neighbouring cluster. Figure 24 shows the two mean distances. For the sample drawn there the mean distance to the nearest other cluster is more than twice the mean distance within its own cluster, so its silhouette lies above one half.

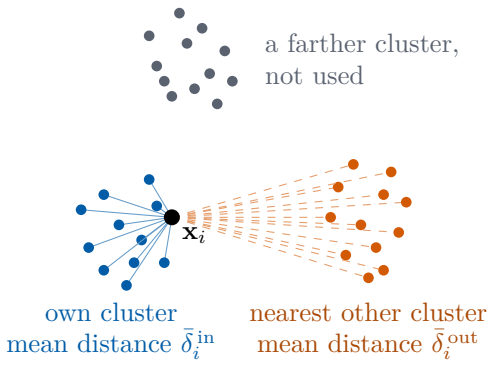


Figure 24: The two mean distances of the silhouette (4.9) for one sample $\mathbf{x}_i$ (black). Solid lines join it to the other samples of its own cluster, whose mean length is $\bar{\delta}_i^{\text{in}}$, dashed lines to the samples of the nearest other cluster, whose mean length is $\bar{\delta}_i^{\text{out}}$. Clusters farther away do not enter. Schematic arrangement.

The mean silhouette over all samples is computed for each k, and the k with the largest mean is selected. On the 600 points of four groups, both criteria indicate four clusters (Figure 25). The objective bends at $k = 4$ and the mean silhouette is largest there.

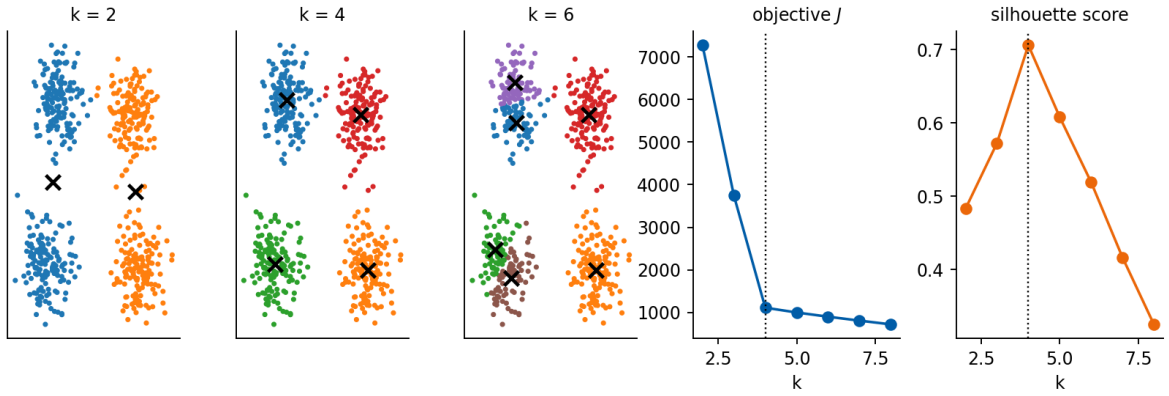


Figure 25: Choosing k for 600 points drawn around four centres. Left, k-means with 2, 4 and 6 clusters. Right, the objective J and the mean silhouette (4.9) for $k = 2, \ldots, 8$. The objective bends at $k = 4$, and the silhouette has its maximum there.

4.4 Clusters of digits

On MNIST we ask k-means for ten clusters, once on the 50 principal components of the 5000 test digits and once on their t-SNE map, and compare the clusters with the digits. Table 6 shows that the adjusted Rand index is 0.384 on the principal components and 0.762 on the t-SNE map. The index has no absolute scale between its two reference values, zero for chance and one for identical partitions, and it is read by comparison. The value 0.384 shows that the clusters found on the principal components carry part of the digit structure, far above chance, but they are far from the ten digits, as the comparison with the t-SNE map shows. Clustering is much easier on the map, where the digits already form separate islands, than in the 50-dimensional space, where they form overlapping, non-spherical sets of the kind k-means handles poorly. When we choose k on the t-SNE map by the silhouette, without using the labels, the objective decreases smoothly without a clear elbow, and the mean silhouette rises to a plateau between 8 and 11 clusters with its maximum at 11, close to but not equal to the number of digits.

Table 6: k-means with $k = 10$ on 5000 MNIST test digits, compared with the true digits.

input of k-means	ARI
50 principal components of the pixels	0.384
t-SNE map of the 50 components	0.762

4.5 Clusters of arbitrary shape

The Voronoi cells of k-means are convex, so k-means cannot follow clusters that are curved or interlocked. Two interlocking half-moons of points, in Figure 27, are cut straight across by k-means with two clusters, with an adjusted Rand index of 0.25 against the two moons. Density-based clustering, of which DBSCAN (density-based spatial clustering of applications with noise) [19] is the standard example, defines a cluster as a connected region of high sample density and needs neither the number of clusters nor any assumption on their shape.

Definition 4.1 (DBSCAN)**.** Let $\varepsilon > 0$ be a radius and $n_{\min}$ a minimum number of samples. A sample is a core sample if at least $n_{\min}$ samples, itself included, lie within distance ε of it. A sample that is not a core sample but lies within ε of one is a border sample, and every other sample is noise. A cluster is a maximal set of core samples in which any two are connected by a chain of core samples with consecutive distances at most ε, together with the border samples within ε of these core samples.

The clusters grow outward from dense regions along chains of close samples, so they follow any shape, and isolated samples remain unassigned as noise. On 70 points of two half-moons with 8 scattered points, DBSCAN with $\varepsilon = 0.25$ and $n_{\min} = 4$ finds 3 clusters, with 65 core samples, 4 border samples and 9 noise samples (Figure 26). On the 500 points of the interlocking moons, DBSCAN with $\varepsilon = 0.2$ and $n_{\min} = 5$ recovers both moons with an adjusted Rand index of 1.00.

The price is a sensitivity to the two parameters. A radius that is too small declares most samples noise, a radius that is too large merges all clusters into one, and when the density varies between clusters no single radius suits all of them. A common heuristic sets $n_{\min}$ to about twice the dimension of the data, which gives the value 4 used for the points in the plane, and reads ε from the sorted distances of all samples to their $n_{\min}$-th nearest neighbour,

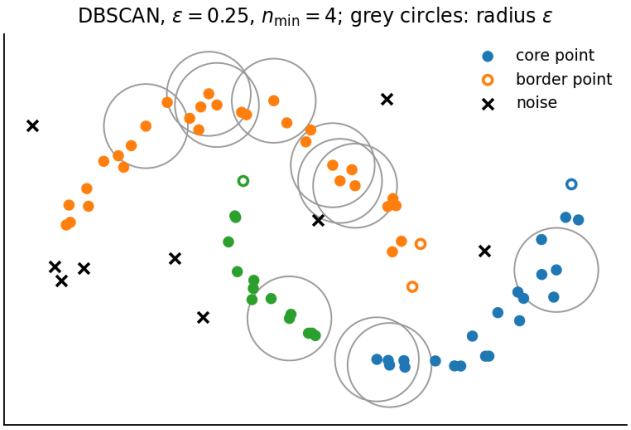


Figure 26: DBSCAN with radius $\varepsilon = 0.25$ and $n_{\min} = 4$ on 78 points. Core samples are filled, border samples open and noise samples marked by crosses. The grey circles of radius ε around some core samples show how the clusters grow along chains of close samples.

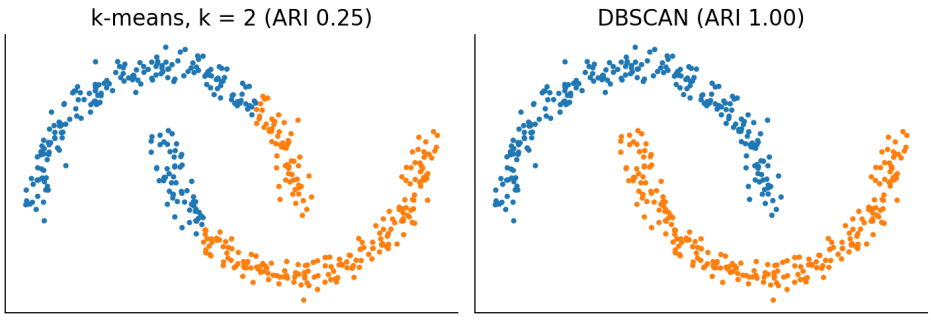


Figure 27: Two interlocking half-moons of 500 points. Left, k-means with two clusters cuts across the moons. Right, DBSCAN with $\varepsilon = 0.2$ and $n_{\min} = 5$ follows them.

at the knee where the curve turns from slow to steep growth and the samples of sparse regions begin. The blood-cell data of Section 6 show this sensitivity.

Both clustering methods so far operate on a representation chosen in advance, the pixels, their principal components or a t-SNE map, and the MNIST results show that the representation matters more than the clustering method. k-means on the t-SNE map doubled the agreement with the digits compared with the principal components. A t-SNE map, however, is a picture of one data set, with no way back to the data and with properties that depend on the perplexity. A representation is needed that is learned from the data, is nonlinear like t-SNE, and is a function like PCA that applies to new samples and can be inverted. The autoencoder provides it.

5 Autoencoders

5.1 PCA as an encoder and a decoder

Equation (2.9) can be read as two maps. The first, $\mathbf{w} = V_d^\top(\mathbf{x} - \bar{\mathbf{x}})$, compresses a sample of D numbers into d numbers. The second, $\hat{\mathbf{x}} = \bar{\mathbf{x}} + V_d\mathbf{w}$, expands the d numbers back into a sample. Both are linear and are fitted to the data by minimising the reconstruction error (2.11). An autoencoder keeps this structure and replaces the two linear maps by neural networks [20, 21]. Because neural networks can represent nonlinear functions, the set of reconstructions is no longer a plane but can be a curved surface that follows the data, as the curve of Figure 11 requires.

Definition 5.1 (Autoencoder)**.** An autoencoder consists of an encoder $f_{\boldsymbol{\phi}} : \mathbb{R}^D \to \mathbb{R}^d$ and a decoder $g_{\boldsymbol{\theta}} : \mathbb{R}^d \to \mathbb{R}^D$, neural networks with parameters $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$. The encoder maps a sample to its latent vector or code $\mathbf{z} = f_{\boldsymbol{\phi}}(\mathbf{x})$, and the decoder maps the code to the reconstruction $\hat{\mathbf{x}} = g_{\boldsymbol{\theta}}(\mathbf{z})$. The space $\mathbb{R}^d$ of codes is the latent space, and $d < D$ is its dimension. The parameters are found by minimising the mean squared reconstruction error over the training samples,

$$\mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{D}\left\|\mathbf{x}_i - g_{\boldsymbol{\theta}}(f_{\boldsymbol{\phi}}(\mathbf{x}_i))\right\|^2 = \frac{1}{nD}\sum_{i=1}^{n}\sum_{j=1}^{D}(x_{ij} - \hat{x}_{ij})^2. \tag{5.1}$$

No label appears in the loss. Each sample is its own target, and the network is trained to reproduce its input. This task would be trivial without the bottleneck, since a network that copies its input achieves zero loss. The bottleneck $d \ll D$ forbids copying. The d numbers of $\mathbf{z}$ must carry whatever the decoder needs to rebuild the sample, and minimising the loss forces the encoder to extract the features of the data that vary most between samples and to discard the rest. The loss compares the input and the reconstruction pixel by pixel, so it rewards getting the bulk of the ink in the right place and pays little attention to features that occupy few pixels.

5.2 A dense autoencoder for digits

The autoencoder we train on MNIST has fully connected layers. Its encoder narrows the input of 784 pixels in three steps to d latent numbers, and its decoder widens them back in

three mirrored steps,

$$\mathbf{h}_1 = \mathrm{ReLU}\left(W_1\mathbf{x} + \mathbf{b}_1\right) \in \mathbb{R}^{256}, \tag{5.2a}$$

$$\mathbf{h}_2 = \mathrm{ReLU}\left(W_2\mathbf{h}_1 + \mathbf{b}_2\right) \in \mathbb{R}^{64}, \tag{5.2b}$$

$$\mathbf{z} = W_3\mathbf{h}_2 + \mathbf{b}_3 \in \mathbb{R}^{d}, \tag{5.2c}$$

$$\mathbf{h}_3 = \mathrm{ReLU}\left(W_4\mathbf{z} + \mathbf{b}_4\right) \in \mathbb{R}^{64}, \tag{5.2d}$$

$$\mathbf{h}_4 = \mathrm{ReLU}\left(W_5\mathbf{h}_3 + \mathbf{b}_5\right) \in \mathbb{R}^{256}, \tag{5.2e}$$

$$\hat{\mathbf{x}} = \mathrm{sigm}\left(W_6\mathbf{h}_4 + \mathbf{b}_6\right) \in (0,1)^{784}, \tag{5.2f}$$

where $\mathrm{ReLU}(a) = \max(0, a)$ and the logistic function $\mathrm{sigm}(a) = 1/(1 + e^{-a})$ act on each component. The encoder parameters are $\boldsymbol{\phi} = \{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3\}$ and the decoder parameters $\boldsymbol{\theta} = \{W_4, \mathbf{b}_4, W_5, \mathbf{b}_5, W_6, \mathbf{b}_6\}$. Figure 28 draws the network.

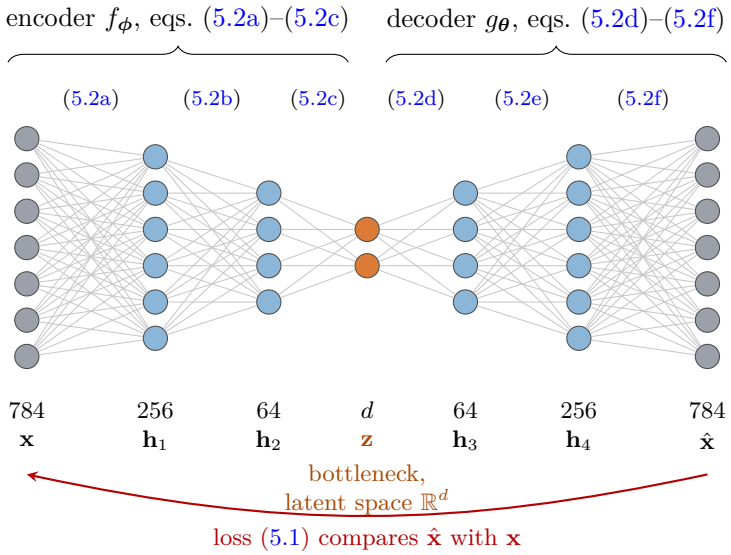


Figure 28: The dense autoencoder of equations (5.2). The widths of the layers, written under the columns, narrow from the 784 pixels of $\mathbf{x}$ through 256 and 64 hidden units to the d latent numbers $\mathbf{z}$ (orange) and widen back to the 784 pixels of $\hat{\mathbf{x}}$. Every grey line stands for a weight. Only a few neurons per layer are drawn. The loss (5.1) compares the output with the input, so no label enters the training.

The information flows through Figure 28 from left to right. The image enters as a vector of 784 grey values. The first layer (5.2a) forms 256 weighted sums of all pixels, each row of W_1 being a pattern of pixels that the unit responds to, and the rectifier keeps only the positive responses. The second layer (5.2b) combines these 256 responses into 64 features of a higher order. The third layer (5.2c) is linear and produces the code $\mathbf{z}$, the only information about the image that is passed on. It has no activation function, so that the code can take any real values. The decoder reverses the narrowing. Equation (5.2d) expands the d numbers into 64 features, equation (5.2e) into 256, and equation (5.2f) produces 784 values that the logistic function confines to the interval $(0, 1)$ of the pixel values. The loss (5.1) then compares $\hat{\mathbf{x}}$ with the input, and its gradient with respect to all twelve parameter arrays is obtained by backpropagation from the output back through the decoder, the code and the encoder.

A dense layer has one weight for every pair of an input unit and an output unit and one bias for every output unit, so the first layer alone has

$$784 \cdot 256 + 256 = 200\,960 \tag{5.3}$$

parameters. Table 7 lists all layers for $d = 16$. The first and the last layer hold 92% of the 437 664 parameters, and the layers near the bottleneck are cheap. All parameters are trained together by the Adam optimiser with learning rate 10^{-3}, in minibatches of 256 images drawn from the 60 000 training images, for 15 passes over the data. The test images are not used for training. They serve to measure the reconstruction error on images the network has not seen.

Table 7: Layers and parameters of the dense autoencoder of equations (5.2) with $d = 16$ latent numbers. ReLU and the logistic output function have no parameters.

part	equation	output width	parameters
encoder	(5.2a)	256	200 960
	(5.2b)	64	16 448
	(5.2c)	16	1 040
decoder	(5.2d)	64	1 088
	(5.2e)	256	16 640
	(5.2f)	784	201 488
total			437 664

After training, the network is used in three ways. The encoder alone maps any image, also a new one, to its code $\mathbf{z}$, which serves as a compressed representation for visualisation and clustering. The decoder alone maps a code back to an image. The composition of both gives the reconstruction, and the reconstruction error of a sample, the quantity used for anomaly detection in Section 7, measures how well the sample fits what the network has learned.

5.3 A linear autoencoder recovers the principal subspace

When the layers are linear, the autoencoder can be solved in closed form. Let the encoder and the decoder be affine maps, $\mathbf{z} = A\mathbf{x} + \mathbf{a}$ and $\hat{\mathbf{x}} = B\mathbf{z} + \mathbf{b}$, with $A \in \mathbb{R}^{d \times D}$ and $B \in \mathbb{R}^{D \times d}$. The reconstruction is

$$\hat{\mathbf{x}} = BA\,\mathbf{x} + \mathbf{c}, \qquad \mathbf{c} = B\mathbf{a} + \mathbf{b}. \tag{5.4}$$

The constant $\mathbf{c}$ enters the loss as the intercept of a least-squares problem, and setting the derivative of $\sum_i \|\mathbf{x}_i - BA\mathbf{x}_i - \mathbf{c}\|^2$ with respect to $\mathbf{c}$ to zero gives $\mathbf{c} = \bar{\mathbf{x}} - BA\bar{\mathbf{x}}$. With this optimal constant, $\hat{\mathbf{x}}_i - \bar{\mathbf{x}} = BA(\mathbf{x}_i - \bar{\mathbf{x}})$, and the loss becomes a function of the product $M = BA$ alone,

$$\mathcal{L} = \frac{1}{nD}\|X_c - X_c M^\top\|_F^2, \qquad \operatorname{rank} M \le d, \tag{5.5}$$

where the rank bound holds because M factors through the d-dimensional latent space. The matrix $X_c M^\top$ has rank at most d, so the Eckart–Young theorem (2.19), applied to $F = X_c$, bounds the loss by $\frac{1}{nD}\sum_{l>d} s_l^2 = \frac{1}{D}\sum_{l>d}\lambda_l$, the PCA error (2.14). The bound is attained for $M = V_d V_d^\top$, since $X_c V_d V_d^\top = X_d$. Every factorisation of this M is a global minimum,

$$A = G\,V_d^\top, \qquad B = V_d\,G^{-1}, \qquad G \in \mathbb{R}^{d \times d} \text{ invertible}, \tag{5.6}$$

and when $\lambda_d > \lambda_{d+1}$ the uniqueness part of the theorem shows that every global minimum has $X_c M^\top = X_d$. The rows of $X_c M^\top = X_c A^\top B^\top$ lie in the column space of B, which has dimension at most d, and the rows of X_d span the principal subspace of dimension d. The column space of B is therefore the principal subspace.

Linear autoencoder and PCA. *A linear autoencoder with d latent units trained by the squared reconstruction error reaches its global minimum when its decoder spans the subspace of the first d principal directions. Its codes are then an invertible affine transformation* $\mathbf{z} =$

$G\,\mathbf{w}+\mathbf{c}'$ of the PCA scores, with a constant vector $\mathbf{c}'$. Gradient descent on this loss has no local minima other than the global one, and all other stationary points are saddle points [22, 23].

A linear autoencoder learns the same subspace as PCA, so an autoencoder can only do better than PCA through its nonlinearities. The individual latent coordinates of a linear autoencoder, however, are not the principal components. The matrix G in equation (5.6) is arbitrary, so the latent axes need be neither orthogonal nor ordered by variance, and they mix the principal directions. The loss determines the subspace and leaves the coordinates within it free. The same freedom is present in a nonlinear autoencoder, where any invertible transformation of the latent space can be absorbed into the encoder and the decoder, so the individual latent coordinates of an autoencoder have no intrinsic meaning. Figure 29 contrasts the linear and the nonlinear case.

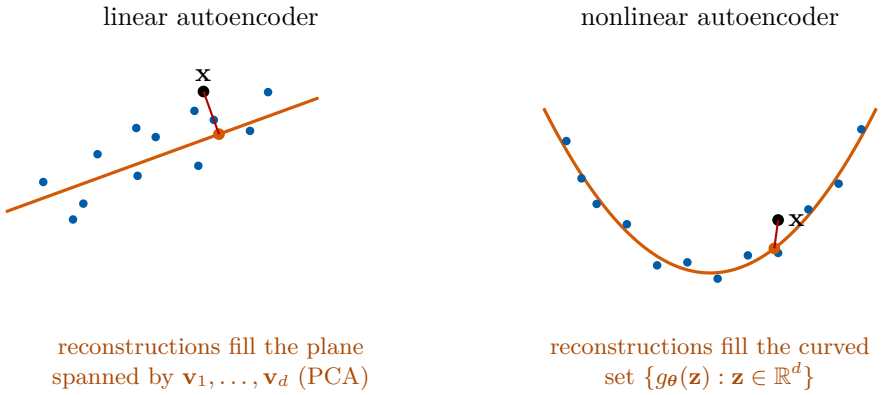


Figure 29: What the decoder can produce. Left, a linear autoencoder reconstructs every sample on the plane of the first d principal directions (orange line), and its optimum coincides with PCA. Right, a nonlinear decoder produces reconstructions on a curved set of dimension d, which can follow data that lie near a curved set. In both cases the reconstruction error (red) measures the distance of a sample from the set of possible reconstructions.

5.4 Reconstructions and latent codes of digits

We train the dense autoencoder twice, with $d = 2$ latent numbers, so that the latent space can be plotted, and with $d = 16$. On the 5000 test images the mean squared error per pixel is 0.0404 for $d = 2$ and 0.0134 for $d = 16$ (Table 8). Figure 30 shows reconstructions. With sixteen latent numbers, 49 times fewer than pixels, the reconstructions are blurred versions of the inputs with the correct digit and shape. With two numbers many digits are reconstructed as a different digit, a 3 as a blurred 5 or 8 and a 7 and a 4 as 9s, because two numbers do not suffice to distinguish ten digits together with their styles.

The pixel-wise error shows where the reconstruction fails. Figure 31 takes the test digit 3 that the autoencoder with $d = 16$ reconstructs best and displays the squared difference $(x_j - \hat{x}_j)^2$ of each pixel. The errors lie along the edges of the strokes, where the reconstruction is blurred, and the mean of the map, 0.0052, is the loss of this image. The 16 latent numbers of this digit range from -21.81 to 13.37. These values have no fixed scale, since nothing in the loss (5.1) constrains the size of the codes.

The codes themselves form a new data set. Figure 32 plots the two-dimensional codes of the 5000 test digits. Each image is mapped by the encoder to one point $\mathbf{z}_i = f_{\phi}(\mathbf{x}_i)$, so the data set becomes a cloud of 5000 isolated points in the latent plane. The codes of the 1s form an

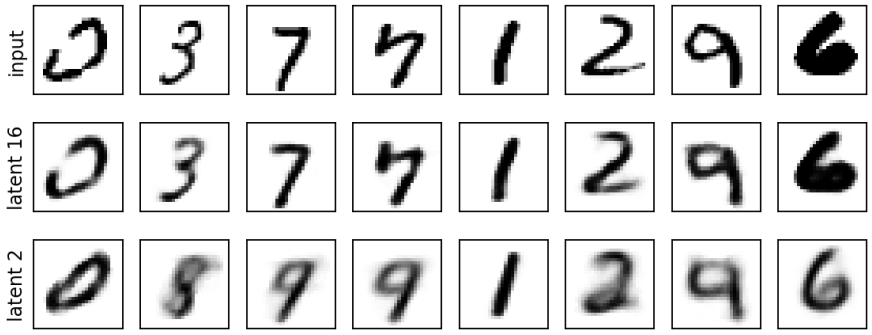


Figure 30: Eight MNIST test digits (top) and their reconstructions by the dense autoencoder with $d = 16$ (middle) and $d = 2$ (bottom) latent numbers.



Figure 31: A test digit $\mathbf{x}$, its reconstruction $\hat{\mathbf{x}}$ by the autoencoder with $d = 16$, and the squared error of each pixel. The mean of the error map, 0.0052, is the loss (5.1) of this image.

arm extending to the right, the 0s lie in the upper left, and the other digits overlap in the centre. Similar images receive nearby codes, because the encoder is a continuous function and the decoder must rebuild similar images from similar codes. The cloud has an irregular shape, its extent along the two axes differs, and large regions of the plane contain no code at all, for example the upper right quadrant and the lower left corner. The loss (5.1) was evaluated only at the codes of training images, and the region between them was never part of the training.

With $d = 16$ the latent space cannot be plotted directly, but it can be analysed with the methods of the previous sections (Figure 33). PCA of the 16-dimensional codes superposes the digits, as PCA of the pixels did, while the t-SNE map of the codes separates them into islands. The latent space therefore contains the distinction between digits, but arranged along curved sets that a linear projection does not separate. Table 8 quantifies this with k-means. Directly on the 16 latent numbers k-means agrees with the digits with an adjusted Rand index of 0.284, less than the 0.384 obtained on 50 principal components of the pixels, and on the t-SNE map of the codes it reaches 0.669. The loss rewards a good reconstruction and nothing else, so it does not ask for codes that form compact, round clusters, and k-means applied directly to the codes is not guaranteed to find the digits.

The encoder of equations (5.2) treats an image as an unordered list of 784 numbers and must learn from the data that neighbouring pixels belong together. A convolutional encoder, built from the convolutional layers of the earlier lectures, compares neighbouring pixels by construction. Strided convolutions reduce the image to a few small feature maps, a dense layer maps them to the code, and the decoder reverses these steps with transposed convolutions, layers that enlarge a feature map by the stride and so undo the reduction of a strided convolution. Table 8 compares such encoders with the dense one. With two latent numbers the convolutional autoencoder reconstructs about as well as the dense one, and its codes separate the digits better. A deeper convolutional autoencoder with sixteen latent numbers, which also

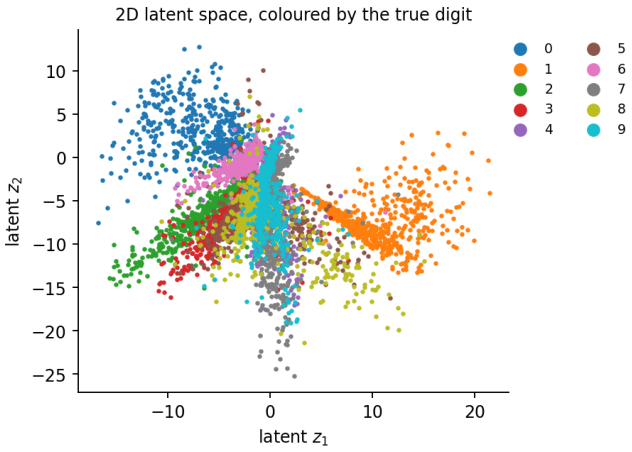


Figure 32: Two-dimensional latent codes $\mathbf{z}_i = f_\phi(\mathbf{x}_i)$ of 5000 MNIST test digits from the dense autoencoder with $d = 2$, coloured by the true digit. Each image becomes one point. The codes form a cloud of irregular shape with dense and sparse parts, and regions such as the upper right and the lower left contain no code.

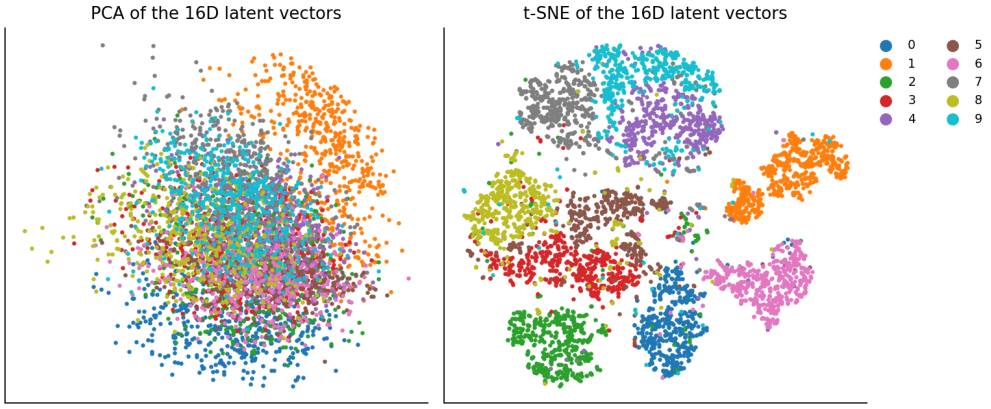


Figure 33: The 16-dimensional codes of 5000 MNIST test digits (dense autoencoder, $d = 16$). Left, projection onto their first two principal components. Right, their t-SNE map. Colours show the true digits.

uses batch normalisation, a rescaling of each channel to zero mean and unit variance over the minibatch followed by a learned scale and shift, halves the reconstruction error of the dense autoencoder and gives the best organised latent space of the table. A better encoder improves the reconstruction and the organisation of the latent space together, although the digits were never part of the loss.

Table 8: Autoencoders on MNIST. The test error is the loss (5.1) on 5000 test images. The last two columns give the adjusted Rand index of k-means with $k = 10$ against the true digits, applied to the codes directly and to their t-SNE map. The last row gives the same indices for 50 principal components of the pixels. The convolutional autoencoder with $d = 2$ has two convolutions with stride 2 ($28 \to 14 \to 7$ pixels, 32 and 64 channels) and 844 291 parameters. The deep convolutional autoencoder with $d = 16$ has three convolutions with 32, 64 and 128 channels, batch normalisation and 3 412 113 parameters.

representation	d	test error	ARI, codes	ARI, t-SNE of codes
dense autoencoder	2	0.0404	0.197	–
convolutional autoencoder	2	0.0389	0.269	–
dense autoencoder	16	0.0134	0.284	0.669
deep convolutional autoencoder	16	0.0068	0.566	0.912
PCA of the pixels	50	–	0.384	0.762

A good reconstruction does not by itself give interpretable codes

The reconstruction loss constrains the composition $g_{\boldsymbol{\theta}} \circ f_{\boldsymbol{\phi}}$ and leaves the latent coordinates free up to any invertible transformation, as equation (5.6) shows in the linear case. Groups in the data may therefore appear in the latent space along curved or elongated sets, and their separation must be examined with tools such as t-SNE or clustering. The latent space is a learned representation of the data, and its structure must be checked against independent information before it is interpreted.

6 Blood cells without labels

The methods developed on the digits are now applied together to the first scientific question, whether the eight types of blood cells of Section 1.2 appear as groups when the images are analysed without their types. We compare three representations of the 3421 test images, the 50 principal components of the pixel values, the t-SNE map of these components, and the code of a convolutional autoencoder trained on the training images alone. The autoencoder has the structure of the convolutional autoencoder of Section 5.4, adapted to three colour channels and sixteen latent numbers, and it is trained by Adam with the loss (5.1). Table 9 lists its settings.

Table 9: The convolutional autoencoder for the blood-cell images.

encoder	convolutions with stride 2, $28 \to 14 \to 7$ pixels, 32 and 64 channels, dense layers of 128 units and 16 latent numbers
decoder	mirror image of the encoder with transposed convolutions
parameters	849 043
training	11 959 training images, 20 passes, minibatches of 128, about two minutes on one processor

Figure 34 shows the three maps. The platelets form a separate group in all of them, and the erythroblasts form a group next to the platelets. Both differ from the white blood cells in

features that affect many pixels at once, such as the size of the cell. The six types of white blood cells overlap. Neutrophils and eosinophils occupy recognisable regions, and lymphocytes form a band at the edge of the cloud, but basophils, monocytes and immature granulocytes are spread among the other types. The autoencoder codes give a map of the same character as the pixels.

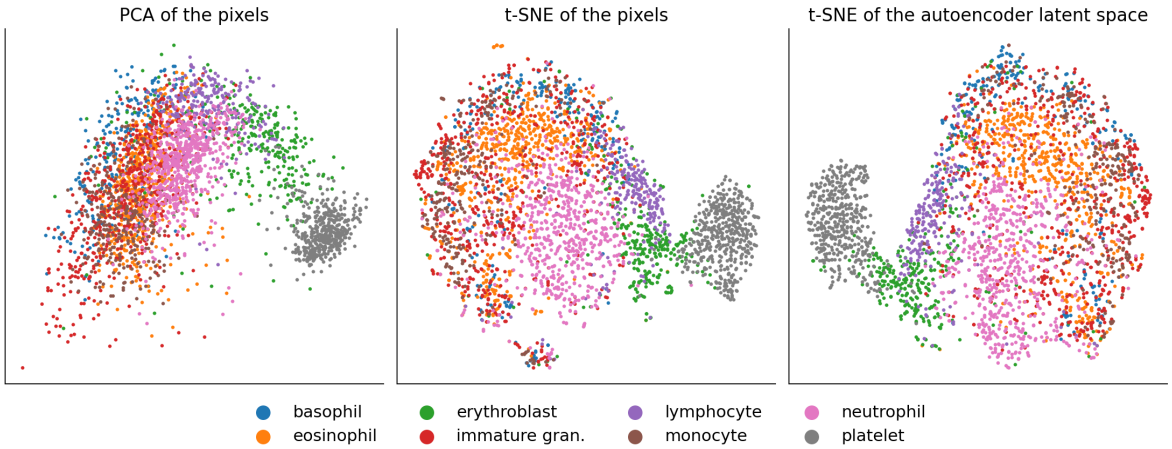


Figure 34: The 3421 BloodMNIST test images, from left to right projected onto the first two principal components of the pixels, mapped by t-SNE from 50 principal components, and mapped by t-SNE from the 16 latent numbers of a convolutional autoencoder trained on the images alone. Colours show the cell types.

k-means with eight clusters, one per type, quantifies the impression. Its adjusted Rand index against the cell types is 0.328 on the 50 principal components of the pixels and 0.336 on the autoencoder codes, so both representations capture the types only partly, and the learned representation is not better than the linear one. To check whether a different choice of map would help, we compute t-SNE maps of three inputs, the principal components of the pixels, the principal components of pixels standardised to unit variance, and the autoencoder codes, each at perplexities 5, 30 and 100. Each map is assessed in two ways. The first is the accuracy of a vote among the 10 nearest neighbours in the map, estimated by five-fold cross-validation. It asks whether the neighbours of a cell in the map share its type. The second is the adjusted Rand index of k-means with eight clusters. Table 10 shows the result.

Table 10: Blood-cell test images. Accuracy of a 10-nearest-neighbour vote (five-fold cross-validation) and adjusted Rand index of k-means with eight clusters, measured on each input before t-SNE and on its t-SNE maps at three perplexities. The chance level of the neighbour vote, the fraction of the most frequent type, is 0.195.

	before t-SNE		perplexity 5		perplexity 30		perplexity 100	
input of t-SNE	vote	ARI	vote	ARI	vote	ARI	vote	ARI
pixels, 50 components	0.713	0.328	0.645	0.255	0.660	0.308	0.648	0.310
standardised pixels, 50 components	0.652	0.255	0.582	0.251	0.607	0.302	0.607	0.304
autoencoder codes, 16 numbers	0.697	0.336	0.644	0.281	0.667	0.316	0.657	0.317

No setting helps. On every input the neighbour vote is more accurate before t-SNE than on any of its maps, because the map preserves the neighbourhoods only approximately. The k-means index stays between 0.25 and 0.34. The neighbour vote is nevertheless far above chance, so that neighbouring cells often share a type, while the types do not form separate groups. The cells form a continuum in which each type occupies a region with soft boundaries.

The groups in the maps have irregular shapes and touch each other, which is the situation for which density-based clustering was designed. We run DBSCAN on the t-SNE map of the pixel components with $n_{\min} = 10$ and several radii, and score the clusters against the types on the cells that are not declared noise (Table 11). No radius gives a useful result. A small radius declares most cells noise, an intermediate radius fragments the map into 67 small clusters with a third of the cells as noise, and larger radii merge the map into a few large clusters unrelated to the types. The density in the map does not follow the cell types.

Table 11: DBSCAN with $n_{\min} = 10$ on the t-SNE map of the blood-cell pixels. The adjusted Rand index is computed on the cells assigned to clusters.

radius ε	clusters	fraction of noise	ARI on clustered cells
1.5	23	0.910	0.22
2.0	67	0.350	0.33
2.5	12	0.072	0.00
3.0	6	0.026	0.00

The negative result has an explanation in terms of what each method measures. All representations used here are built on the squared difference of pixel values, either directly, through PCA and t-SNE, or through the reconstruction loss (5.1) of the autoencoder. Such a measure is dominated by features that change many pixels at once, such as the size of the cell and its overall colour. These features separate platelets and erythroblasts from the white cells. The types of white cells are defined by pathologists through the shape and the lobes of the nucleus and the colour and size of the granules in the cytoplasm, which occupy a few pixels at a resolution of 28×28 and contribute little to a pixel-wise distance. A supervised network trained on the labels learns to give these few pixels a large weight, while an unsupervised method has no reason to. The clusters that an unsupervised method finds are the groups of its own measure of similarity, and they coincide with the categories of an expert only when the expert's distinctions dominate that measure.

7 Anomaly detection

7.1 Reconstruction error as a score of typicality

The third question of unsupervised learning asks which samples are unusual. In the language of Section 1 an anomaly is a sample from a region where the data distribution $p(\mathbf{x})$ is small. The examples of anomalies are usually rare, varied and partly unknown. Fraudsters change their methods, a new kind of tumour has by definition not been seen before, and a faulty measurement can fail in many ways. A classifier trained on known anomalies recognises only those. A method that learns what normal data look like, and flags whatever does not fit, needs no examples of anomalies at all.

An autoencoder supplies such a method. It is trained on normal samples only, and its decoder learns to produce samples near the set where the normal data lie, the curved set of Figure 29. A new sample similar to the training data is mapped to a code and reconstructed close to itself. A sample unlike the training data is also mapped to some code, but the decoder can only produce what it has learned, so the reconstruction resembles normal data and lies far from the input. The squared reconstruction error

$$a(\mathbf{x}) = \frac{1}{D}\left\|\mathbf{x} - g_{\boldsymbol{\theta}}(f_{\boldsymbol{\phi}}(\mathbf{x}))\right\|^2 \tag{7.1}$$

is therefore a score of how unusual $\mathbf{x}$ is, approximately its squared distance from the set of reconstructions. A sample is declared an anomaly when its score exceeds a threshold τ,

$$\text{anomaly} \iff a(\mathbf{x}) > \tau. \tag{7.2}$$

The threshold is fixed with normal data alone, for example as the value exceeded by 5% of normal validation samples, which fixes the rate of false alarms at 5%. Figure 35 illustrates the geometry. For PCA the corresponding score is the squared distance from the principal subspace, $a_{\mathrm{PCA}}(\mathbf{x}) = \frac{1}{D}\|(\mathbb{1} - V_d V_d^\top)(\mathbf{x} - \bar{\mathbf{x}})\|^2$, which by Section 5.3 is what a linear autoencoder computes.

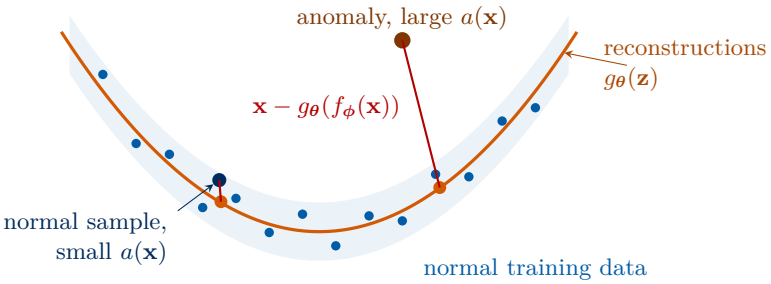


Figure 35: Anomaly detection by reconstruction. The autoencoder is trained on normal samples (blue), and its reconstructions lie on a curved set near them (orange). A normal test sample is reconstructed nearby and has a small error. An anomaly is mapped back onto the set of normal-looking reconstructions and has a large error (red segment), equation (7.1).

7.2 Two anomalies among digits

Two experiments on the digits show the range of difficulty. In the first, the autoencoder with $d = 16$ is trained on all training digits except the 7s, so that every 7 is a kind of image the network has never seen, and we compute the reconstruction error of all 10 000 test digits. In the second, the autoencoder trained on all digits receives photographs from the CIFAR-10 collection [24], converted to grey values and reduced to 28×28 pixels. Figure 36 shows a typical case of each kind, the digit and the photograph whose errors are the medians of their groups. The digit is reconstructed well. The photograph is reconstructed as a blurred, digit-like shape, since the decoder can produce nothing else, and its error is about 17 times larger.

The two anomalies behave very differently (Table 12 and Figure 37). The photographs are almost all flagged by the threshold that flags 5% of the digits. The unseen 7s have a larger mean error than the digits of the trained classes, but the two distributions overlap strongly, and the same threshold flags only 12.9% of them. A 7 shares its strokes with the 1s and 9s, so the network reconstructs many 7s well although it has never seen one. Among the 100 test digits with the largest error, 24 are 7s, against the 10% share of 7s in the test set. The ranking concentrates the 7s at the top, but most of the top-ranked images are poorly written digits of the trained classes, which are unusual in their own way.

An anomaly far from the training data, such as a photograph among digits, is easy to detect. An anomaly that resembles the training data is hard, and the quality of a detector depends on the threshold. Comparing two scores therefore requires a measure of quality that does not depend on a particular threshold.

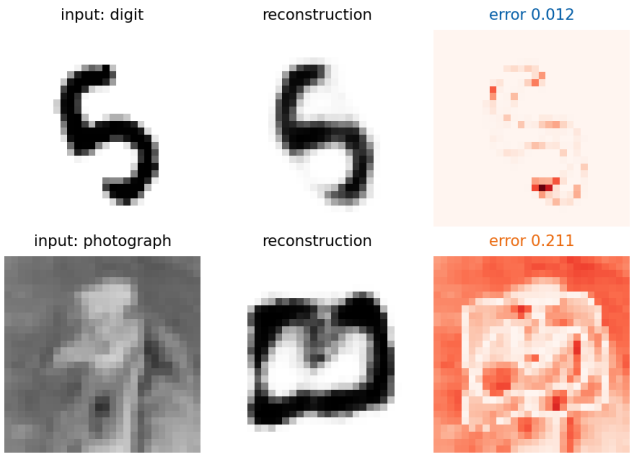


Figure 36: Autoencoder trained on all MNIST digits. Top, a test digit, its reconstruction and the squared error of each pixel. Bottom, the same for a CIFAR-10 photograph converted to 28×28 grey values. Each is the median case of its group, with reconstruction errors 0.0121 and 0.2112.

Table 12: Anomaly detection among digits with the autoencoder of $d = 16$. For the 7s it is trained on all other digits, for the photographs on all digits. The threshold flags 5% of the normal test digits. The area under the ROC curve (AUC) is defined in Section 7.5.

	unseen 7s	photographs
mean error of normal digits	0.0136	0.0134
mean error of anomalies	0.0187	0.2271
anomalies flagged at 5% false alarms	0.129	0.998
AUC	0.678	0.999

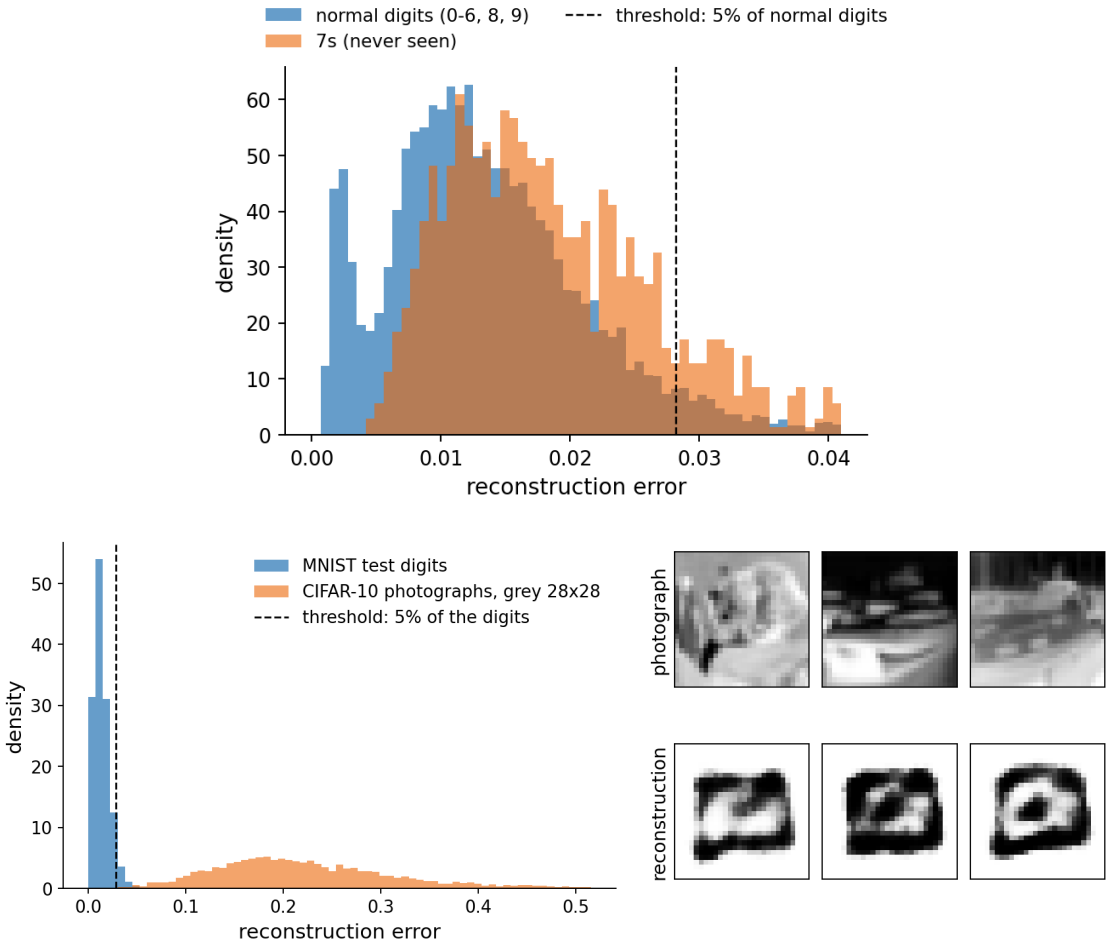


Figure 37: Distributions of the reconstruction error. Top, autoencoder trained without the 7s, with the errors of the trained digits (blue), of the unseen 7s (orange) and the threshold that flags 5% of the trained digits (dashed). Bottom, autoencoder trained on all digits, with the errors of the test digits (blue) and of CIFAR-10 photographs (orange), and three photographs with their digit-like reconstructions.

7.3 Threshold, hits and false alarms

Every score $a(\mathbf{x})$ orders the samples along one axis, and a threshold τ divides this axis into flagged and unflagged samples. For a test set with N_{A} anomalies and N_{N} normal samples, the threshold produces four counts, collected in the confusion matrix of Table 13, the true positives $\mathrm{TP}(\tau)$ (anomalies flagged), the false negatives $\mathrm{FN}(\tau)$ (anomalies missed), the false positives $\mathrm{FP}(\tau)$ (normal samples flagged) and the true negatives $\mathrm{TN}(\tau)$.

Table 13: The confusion matrix of a threshold τ on an anomaly score.

	flagged, $a(\mathbf{x}) > \tau$	not flagged, $a(\mathbf{x}) \le \tau$
anomaly (N_{A} samples)	true positive $\mathrm{TP}(\tau)$	false negative $\mathrm{FN}(\tau)$
normal (N_{N} samples)	false positive $\mathrm{FP}(\tau)$	true negative $\mathrm{TN}(\tau)$

Two rates summarise the matrix, each normalised by the size of one class,

$$\mathrm{TPR}(\tau) = \frac{\mathrm{TP}(\tau)}{N_{\mathrm{A}}}, \qquad \mathrm{FPR}(\tau) = \frac{\mathrm{FP}(\tau)}{N_{\mathrm{N}}}. \tag{7.3}$$

The true-positive rate, also called the recall or the sensitivity, is the fraction of anomalies caught. The false-positive rate is the fraction of normal samples flagged, the rate of false alarms. If the scores of normal samples and anomalies have the densities $p_{\mathrm{N}}(a)$ and $p_{\mathrm{A}}(a)$, the two rates are the areas of these densities above the threshold,

$$\mathrm{TPR}(\tau) = \int_{\tau}^{\infty} p_{\mathrm{A}}(a)\,\mathrm{d}a, \qquad \mathrm{FPR}(\tau) = \int_{\tau}^{\infty} p_{\mathrm{N}}(a)\,\mathrm{d}a, \tag{7.4}$$

shown as the shaded areas in Figure 38. Lowering the threshold catches more anomalies and raises more false alarms. No threshold improves one rate without worsening the other, and the choice of τ is a decision about the relative cost of missed anomalies and false alarms, which the data cannot make.

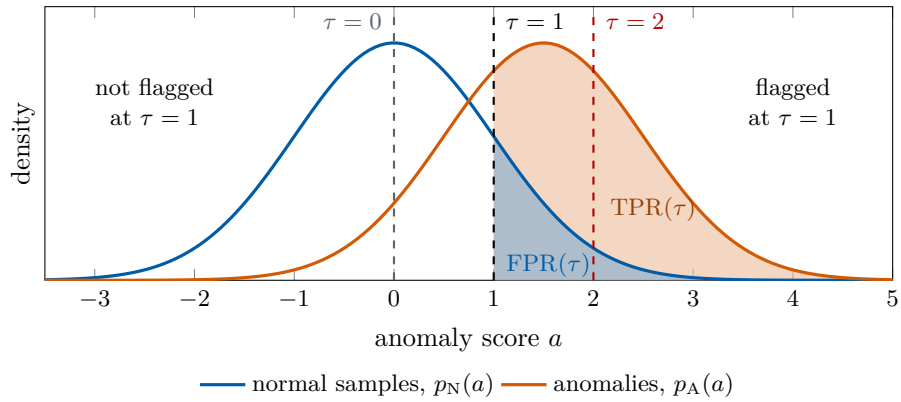


Figure 38: Thresholds on the score. Samples with $a > \tau$ are flagged. For $\tau = 1$ the orange area above τ is the fraction of anomalies caught, $\mathrm{TPR}(\tau)$, and the blue area the fraction of normal samples flagged, $\mathrm{FPR}(\tau)$, equation (7.4). The two normal densities, with means 0 and 1.5 and unit variance, are illustrative. The thresholds $\tau = 2$ and $\tau = 0$ are used in Figure 39.

7.4 The ROC curve

Each threshold gives one pair $(\mathrm{FPR}(\tau), \mathrm{TPR}(\tau))$, one point in the unit square, and lowering τ from above the largest score to below the smallest moves the point from $(0, 0)$, where nothing

is flagged, to $(1, 1)$, where everything is flagged. The curve traced in this way is the receiver operating characteristic (ROC) curve [25], a name that comes from the analysis of radar receivers. It displays all decision rules that the score allows. Figure 39 constructs it for the illustrative densities of Figure 38. Each of the three thresholds marked in Figure 38 gives one point of the curve, and the curve is the set of all such points.

A score that carries no information about the class gives the same distribution to anomalies and normal samples, so $\mathrm{TPR}(\tau) = \mathrm{FPR}(\tau)$ for every threshold, and its ROC curve is the diagonal. A perfect score places all anomalies above all normal samples, and some threshold between them gives $\mathrm{TPR} = 1$ at $\mathrm{FPR} = 0$, so its curve runs along the left and upper edges of the square. The better a score separates the two classes, the closer its curve comes to the upper left corner.

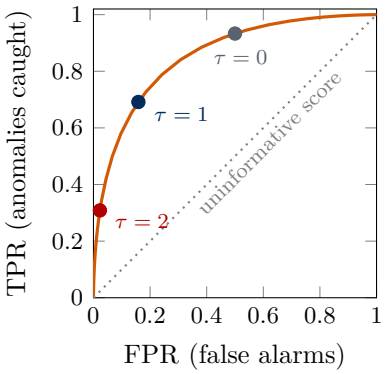


Figure 39: Construction of the ROC curve for the illustrative densities of Figure 38. Each of the three thresholds marked there gives one point $(\mathrm{FPR}, \mathrm{TPR})$ by equation (7.4), and sweeping τ from high to low traces the curve from $(0, 0)$ to $(1, 1)$. The dotted diagonal is the curve of a score without information.

7.5 The area under the curve

The ROC curve compares scores over all thresholds, and a single number is obtained from the area under it, the AUC. The area has a probabilistic meaning that makes it more than a geometric summary [26]. Parametrise the curve by τ. By equation (7.4), $\mathrm{dFPR} = -p_{\mathrm{N}}(\tau)\,\mathrm{d}\tau$, and $\mathrm{FPR} = 0$ at $\tau = +\infty$ and $\mathrm{FPR} = 1$ at $\tau = -\infty$. Substituting and reversing the direction of integration, which removes the minus sign,

$$\begin{aligned} \mathrm{AUC} &= \int_0^1 \mathrm{TPR}\;\mathrm{dFPR} = \int_{+\infty}^{-\infty} \mathrm{TPR}(\tau)\,\big(-p_{\mathrm{N}}(\tau)\big)\,\mathrm{d}\tau \\ &= \int_{-\infty}^{\infty} \mathrm{TPR}(\tau)\,p_{\mathrm{N}}(\tau)\,\mathrm{d}\tau = \int_{-\infty}^{\infty}\int_{\tau}^{\infty} p_{\mathrm{A}}(a)\,p_{\mathrm{N}}(\tau)\,\mathrm{d}a\,\mathrm{d}\tau. \end{aligned} \tag{7.5}$$

The double integral is the probability that an independently drawn anomaly has a larger score than an independently drawn normal sample,

$$\mathrm{AUC} = P\big(a(\mathbf{x}_{\mathrm{anomaly}}) > a(\mathbf{x}_{\mathrm{normal}})\big). \tag{7.6}$$

An uninformative score gives $\mathrm{AUC} = \frac{1}{2}$, the area under the diagonal, and a perfect one gives $\mathrm{AUC} = 1$. On a finite test set the AUC is the fraction of all $N_{\mathrm{A}}N_{\mathrm{N}}$ pairs of an anomaly and a normal sample that the score orders correctly, a quantity known in statistics as the Mann–Whitney statistic. For normal score distributions of unit variance whose means differ

by Δ, the difference of an anomalous and a normal score is normal with mean Δ and variance 2, and equation (7.6) gives

$$\mathrm{AUC} = P(a_\mathrm{A} - a_\mathrm{N} > 0) = \Phi\Big(\frac{\Delta}{\sqrt{2}}\Big), \tag{7.7}$$

where Φ is the standard normal distribution function. The area grows from one half at $\Delta = 0$ towards one as the two distributions separate. Figure 40 shows the area. The AUC requires no threshold and depends only on the order of the scores, so it compares scores, while the choice of the operating point on the curve remains a separate decision.

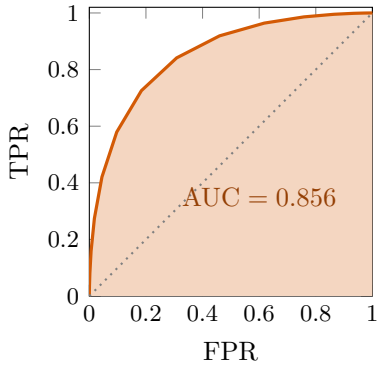


Figure 40: The area under the ROC curve of Figure 39. It equals the probability (7.6) that a randomly chosen anomaly scores higher than a randomly chosen normal sample, here $\Phi(1.5/\sqrt{2}) = 0.856$ for the illustrative densities with $\Delta = 1.5$. The area under the diagonal is $\frac{1}{2}$.

For the unseen 7s the measured ROC curve of Figure 41 has the area 0.678. The point marked on it is the threshold of Table 12, with 5% false alarms and 12.9% of the 7s caught. The score is clearly better than chance, but a randomly chosen 7 outranks a randomly chosen trained digit only in about two cases out of three. For the photographs the area is 0.999, and the curve runs almost along the edges of the square.

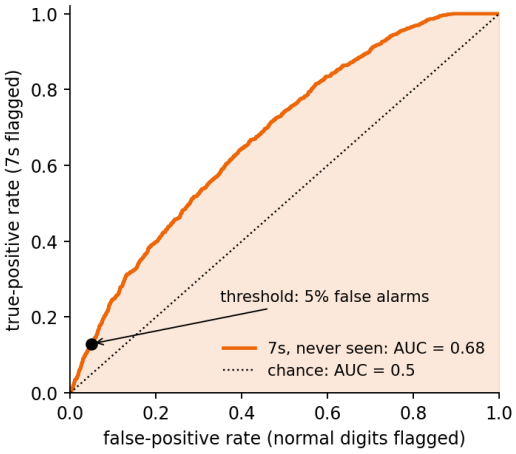


Figure 41: Measured ROC curve of the reconstruction error for the unseen 7s, with the autoencoder trained on all other digits. The area under it is 0.678. The dot marks the threshold that flags 5% of the trained digits and 12.9% of the 7s. The dotted diagonal is the curve of an uninformative score.

7.6 Rare anomalies and misleading accuracy

In real applications anomalies are rare. Among the 284 807 card transactions 492 are fraudulent, a fraction of

$$\pi = \frac{492}{284\,807} = 0.0017. \tag{7.8}$$

This imbalance makes the most familiar measure of a classifier useless. The accuracy, the fraction of correct decisions, of the trivial rule that declares every transaction normal is $1-\pi = 99.83\%$, higher than that of almost any real detector, and the rule catches no fraud at all. The rates (7.3) avoid this trap because each is normalised within one class, so the ROC curve and the AUC do not depend on the proportion of anomalies.

The rates, however, hide what an analyst who has to examine the flagged transactions experiences. The precision, the fraction of flagged samples that are anomalies, follows from the rates and the prevalence π by counting. Of N samples, πN TPR anomalies and $(1-\pi)N$ FPR normal samples are flagged, so

$$\text{precision} = \frac{\text{TP}}{\text{TP}+\text{FP}} = \frac{\pi\,\text{TPR}}{\pi\,\text{TPR} + (1-\pi)\,\text{FPR}}. \tag{7.9}$$

For a rare class the false positives dominate the denominator unless FPR is of the order of π or smaller. A false-positive rate of one percent, which would usually be considered good, flags in a population with $\pi = 0.0017$ several times more normal transactions than there are frauds altogether. Figure 42 shows the consequence for the detector of Section 7.7. It catches three quarters of the frauds, yet only about one flagged transaction in eight is fraudulent.

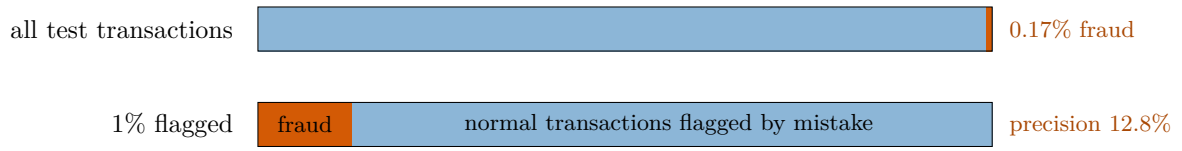


Figure 42: Precision under class imbalance, for the autoencoder of Section 7.7. Top, the test transactions, of which 0.17% are fraudulent (orange sliver, drawn wider than to scale). Bottom, the 1% of transactions with the largest reconstruction errors. The frauds among them are a small part of the flagged set, equation (7.9).

The practically relevant part of the ROC curve is therefore the region of very small false-positive rates, which a linear axis compresses into a thin strip at the left edge. For the transaction data we plot the false-positive rate on a logarithmic scale.

7.7 Fraud in card transactions

We split the transactions at random, keeping the proportion of fraud, into a training part of 80% and a test part of 20%. The test part contains 56 962 transactions, of which 98 are fraudulent. Each of the 29 columns of equation (1.4) is standardised with the mean and standard deviation of the normal training transactions. The autoencoder has the layers $29 \to 16 \to 8 \to 16 \to 29$, with ReLU in the hidden layers and a linear output, since the standardised values are not confined to an interval, and it has only 1253 parameters. It is trained by Adam on the normal training transactions alone, for 10 passes, and the frauds of the training part are never used. On the test part its reconstruction error is the anomaly score. As a baseline we use the mean squared standardised value of the transaction, $\frac{1}{29}\sum_j \left((x_j - \mu_j^{\text{N}})/\sigma_j^{\text{N}}\right)^2$, where μ_j^{N} and σ_j^{N} are the mean and standard deviation of column j

over the normal training transactions. It is the squared distance from the centre of the normal transactions, each column measured in units of its own spread.

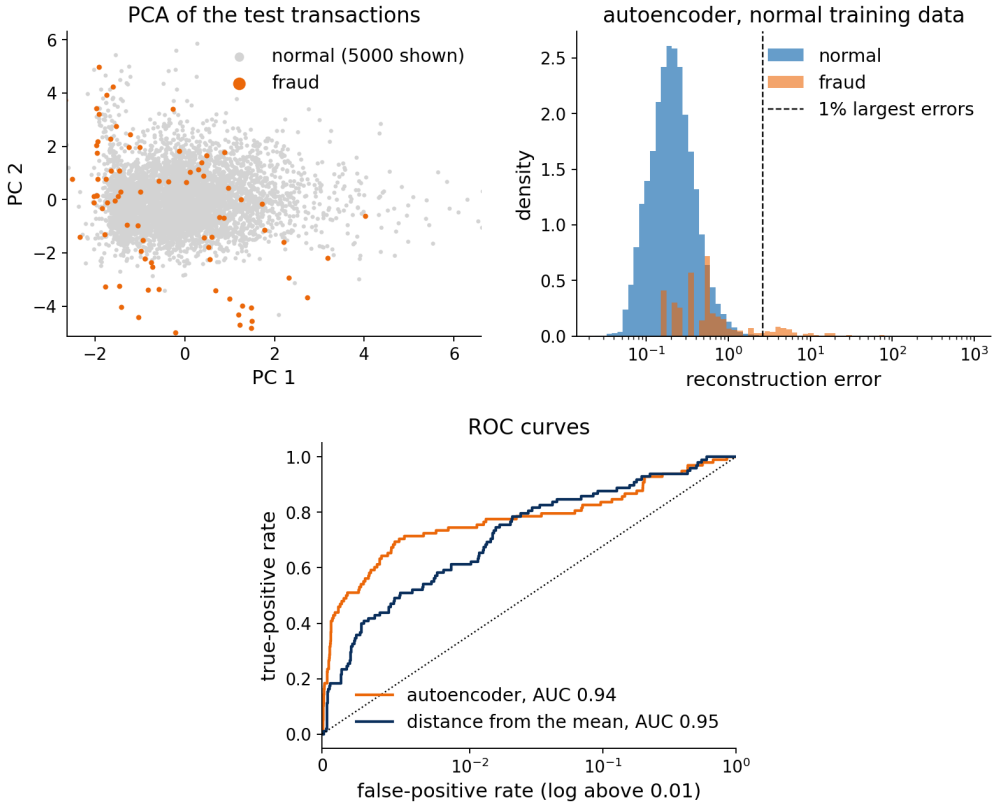


Figure 43: Fraud detection on the 56 962 test transactions. Top left, projection onto the first two principal components of the normal training transactions, which retain 9.9% of the variance, with 5000 normal transactions (grey) and all 98 frauds (orange). Top right, distribution of the reconstruction error of the autoencoder trained on normal transactions (logarithmic axis), with the threshold that flags the 1% largest errors. Bottom, ROC curves of the autoencoder and of the distance from the centre of the normal transactions, with the false-positive rate on a logarithmic scale above 0.01. The areas under the curves are 0.941 for the autoencoder and 0.949 for the distance.

Figure 43 and Table 14 show the result. The reconstruction errors of the frauds split into two parts, as the schematic Figure 44 summarises. About three quarters of the frauds have errors above the threshold that flags 1% of the transactions, far above the peak of the normal transactions, and most of the remaining ones have errors like normal transactions. Flagging the 1% of test transactions with the largest errors catches 73 of the 98 frauds, and 12.8% of the flagged transactions are fraudulent. This is the situation of Figure 42, and an analyst examining the flagged transactions finds about one fraud in eight. Flagging fewer transactions raises the precision and lowers the fraction of frauds caught.

The autoencoder does not beat the simple baseline. Its AUC is 0.941 against 0.949 for the distance from the centre. The ROC curves of Figure 43 show the autoencoder ahead at small false-positive rates and the distance ahead at larger ones, so neither dominates. The reason lies in the data, whose 29 columns are nearly uncorrelated, as Section 2.6 showed, and normal transactions have no low-dimensional structure that a bottleneck of 8 numbers could capture. The autoencoder then reconstructs mainly the part of a transaction near the centre of the normal data, and its error is close to a weighted distance from that centre, which is what the

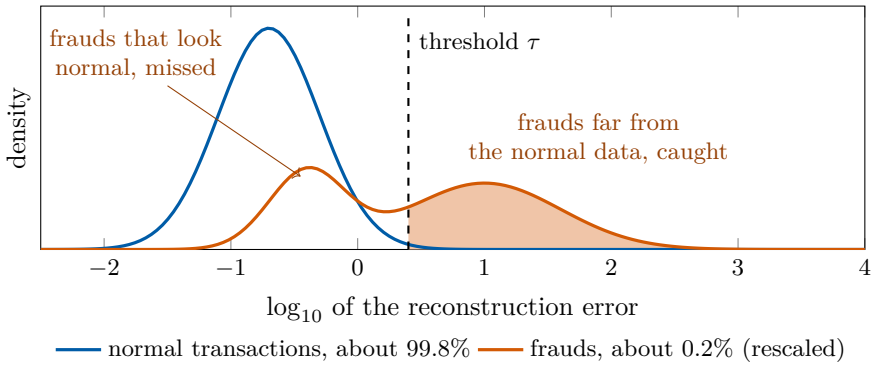


Figure 44: Schematic of the upper right panel of Figure 43, with illustrative shapes. The reconstruction errors of normal transactions form one peak. Part of the frauds lies in a tail far above it and is caught by any reasonable threshold, while another part has errors like normal transactions and is missed at any threshold with few false alarms. The fraud density is drawn on a much larger scale than its share of the data.

Table 14: Fraud detection on the test part (56 962 transactions, 98 frauds). The upper rows flag the stated fraction of transactions with the largest reconstruction error of the autoencoder. The AUC row compares the autoencoder with the distance from the centre of the normal transactions.

fraction flagged	frauds caught	fraction of frauds caught	precision
0.5%	62	0.633	0.218
1%	73	0.745	0.128
2%	76	0.776	0.067
	AUC of the autoencoder 0.941, of the distance from the centre 0.949		

baseline measures directly. A fraud is detectable when some of its standardised values are unusually large. An autoencoder adds value over a distance only when the normal data lie near a curved low-dimensional set, so that a sample can be far from that set while close to the centre.

7.8 An unseen tumour type

For the tumour data the anomaly is a tumour type that was never seen in training. We hold out one of the five types, split the samples of the four remaining types at random into a training part of 80% and a test part of 20%, and score the test samples of the known types together with all samples of the held-out type. From the training part only, we select the 2000 genes with the largest variance and compute their means and standard deviations, with which all samples are standardised. Using the test samples for this selection would leak information about them into the model and make the test optimistic. The autoencoder has the layers $2000 \to 256 \to 32 \to 256 \to 2000$, with ReLU in the hidden layers and 1 042 928 parameters, and is trained by Adam for 300 passes with a small weight decay, a penalty proportional to the squared weights, as in the regularisation of the earlier lectures. The anomaly score is the reconstruction error averaged over the 2000 genes. As a baseline we use the PCA score of Section 7.1, the squared distance from the subspace of the first 32 principal directions of the training part, which equals the error of a linear autoencoder with the same latent dimension. Each of the five types is held out in turn.

Table 15: Detection of a tumour type never seen in training. For each held-out type, the number of training samples, of test samples of the four known types and of samples of the held-out type, and the AUC of the autoencoder and of the PCA reconstruction error with 32 components.

held-out type	training	known test	held-out	AUC autoencoder	AUC PCA
BRCA	400	101	300	0.974	0.985
KIRC	524	131	146	0.998	0.999
LUAD	528	132	141	0.960	0.982
PRAD	532	133	136	0.988	0.992
COAD	578	145	78	0.988	0.991

Both scores recognise the unseen type almost perfectly, with AUC between 0.960 and 0.999 (Table 15). For the colon adenocarcinomas held out, the threshold that flags 5% of the known test samples flags 96% of the COAD samples (Figure 45). PCA is as good as the autoencoder or slightly better for every type. Tumours of different organs differ in the expression of many genes, so an unseen type lies far from the subspace spanned by the known types, and even a linear model recognises it. Moreover, with 400 to 578 training samples, an autoencoder with a million parameters has little data from which to learn a curved structure that would improve on the linear subspace, while PCA has no parameters to fit beyond the subspace itself. A flexible model should be compared with the simplest method that answers the same question, and on small data sets of high dimension the simplest method is often hard to beat.

7.9 When the reconstruction error misleads

The blood cells show that a small reconstruction error does not imply that a sample is familiar. We repeat the experiment of the unseen tumour type with the blood cells. The convolutional autoencoder of Section 6 is trained on seven of the eight cell types for 10 passes, the eighth

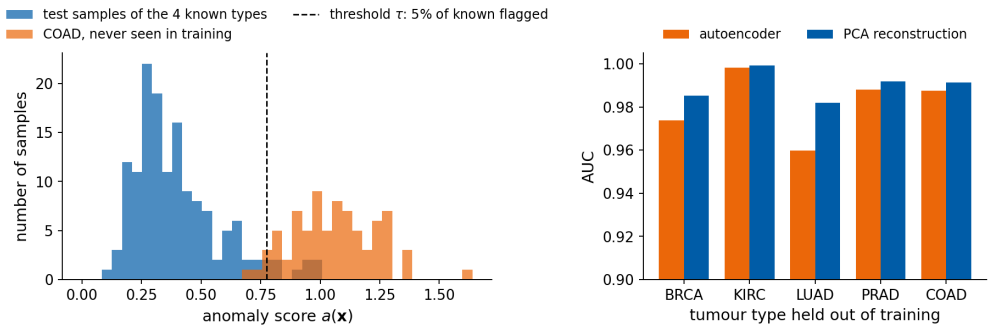


Figure 45: Detection of an unseen tumour type. Left, autoencoder scores of the test samples of the four known types (blue) and of the colon adenocarcinomas (COAD) held out of training (orange), with the threshold τ that flags 5% of the known samples. Right, the AUC of the autoencoder and of the PCA reconstruction error for each held-out type.

type is treated as the anomaly, and the reconstruction error of all test images is scored. Each type is held out in turn (Table 16).

Table 16: Blood cells. AUC of the reconstruction error when one cell type is held out of training and treated as the anomaly. An AUC below 0.5 means that the unseen type is reconstructed better than the trained types.

held-out type	AUC
basophil	0.571
eosinophil	0.644
erythroblast	0.445
immature granulocyte	0.559
lymphocyte	0.522
monocyte	0.585
neutrophil	0.807
platelet	0.335

Only the hidden neutrophils are found with some success, with AUC 0.807, and most other types lie near chance. The platelets and the erythroblasts, which the maps of Section 6 separated most clearly from the rest, have AUC values below one half. Images of a type the network has never seen are reconstructed better than the trained types. A platelet is a small, simple object on a uniform background, and an autoencoder that can rebuild the complex images of white cells rebuilds a simple image easily, whether or not it was part of the training. The reconstruction error measures how difficult a sample is to rebuild with what the network has learned, and this agrees with how improbable the sample is under the distribution of the training data only when unusual samples are also hard to rebuild. A score that measured unusualness directly would be the density $p(\mathbf{x})$ of the normal data, small for a platelet when no platelets were in the training data however simple its image.

8 The latent space as a set of points

The anomaly experiments used the autoencoder as a judge of typicality, and the blood cells showed that the judgement is indirect. The question of what the autoencoder knows about the distribution of the data can be examined directly in its latent space. The encoder is a deterministic function, so each training sample $\mathbf{x}_i$ is mapped to a single code $\mathbf{z}_i = f_\phi(\mathbf{x}_i)$, and

the training set of n samples becomes a set of n isolated points in $\mathbb{R}^d$. The loss (5.1) can be written as a sum over these points,

$$\mathcal{L} = \frac{1}{nD} \sum_{i=1}^{n} \|\mathbf{x}_i - g_{\boldsymbol{\theta}}(\mathbf{z}_i)\|^2, \qquad \mathbf{z}_i = f_{\boldsymbol{\phi}}(\mathbf{x}_i), \tag{8.1}$$

and the decoder enters it only through its values at the n codes. Training adjusts $g_{\boldsymbol{\theta}}$ so that each code is decoded into its own sample. Nothing in equation (8.1) refers to any other point of the latent space. The decoder is a continuous function defined on all of $\mathbb{R}^d$, and between the codes its values are fixed only by continuity and by the form of the network, never by a sample.

Figure 46 shows what this means for the digits. It plots the codes of all 60 000 training images of the autoencoder with $d = 2$ in grey, with the 5000 test codes coloured by digit. The training codes form a cloud with a dense core, arms such as that of the 1s, a sparse rim and empty regions between the arms. When the box that contains the training codes is divided into 50×50 cells, 53.3% of the cells hold at least one training code, and the rest of the box was never reached by the encoder during training. The cloud has no fixed scale or shape either. A different random initialisation of the network can produce a differently shaped cloud with the same reconstruction error, since by the argument following equation (5.6) any invertible transformation of the latent space can be absorbed into the encoder and the decoder. The 16 latent numbers of the digit 3 of Figure 31 range from -21.81 to 13.37, values that are meaningful only relative to the codes of other images. Figure 47 summarises the situation schematically.

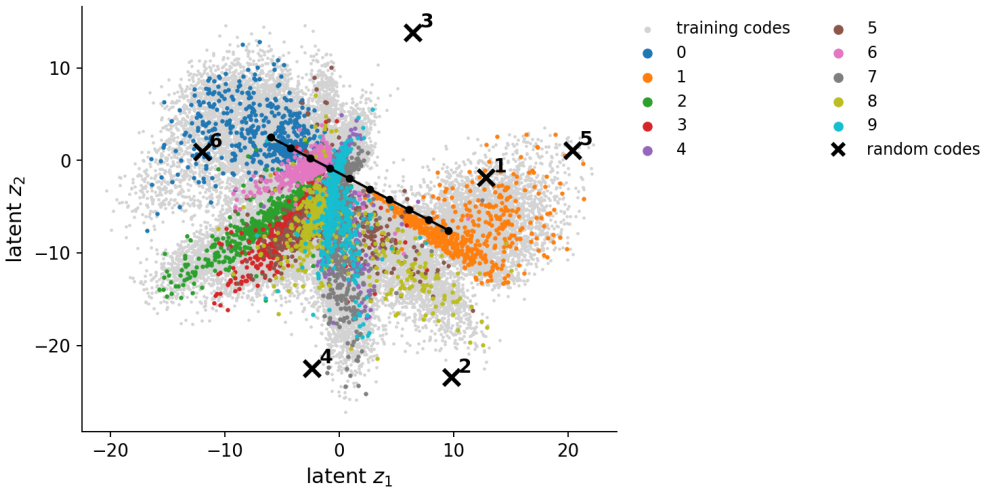


Figure 46: Latent plane of the dense autoencoder with $d = 2$. Grey, the codes of the 60 000 training images, the only points at which the loss was evaluated. Coloured, the codes of 5000 test images by digit. Black dots, ten points on the straight line from the code of a typical 0 to the code of a typical 1. Crosses, six codes drawn uniformly from the box of the training codes. Figure 48 shows the decoded images of the black dots and the crosses.

This picture limits what the autoencoder can do beyond compression. A scientist often wants new samples that resemble the data without being copies of them, for example plausible images of a cell type or intermediate forms between two observed samples. With a decoder at hand, the natural procedure is to choose a code and decode it, and Figure 48 tests the two obvious choices on the digits.

The first choice is a code between the codes of training images, such as the points on the straight line from a typical 0 to a typical 1. The decoded images change from a 0 to a 6, a 9

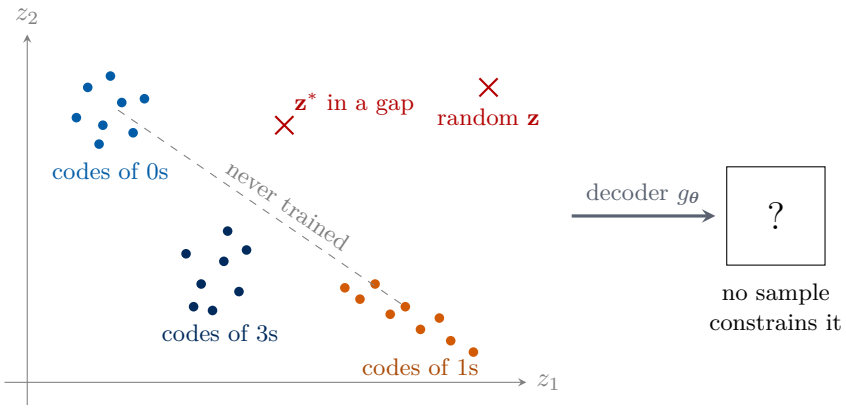


Figure 47: The latent space of an autoencoder as a set of points. Each training sample is mapped to one code, and the codes of similar samples cluster (blue, navy, orange). The loss (8.1) evaluates the decoder only at these codes. A point $\mathbf{z}^*$ in a gap between the clusters, or a code drawn at random, was never part of the training, and nothing determines what the decoder produces there. The shape and scale of the cloud are arbitrary, so there is also no distribution from which a random code could be drawn.

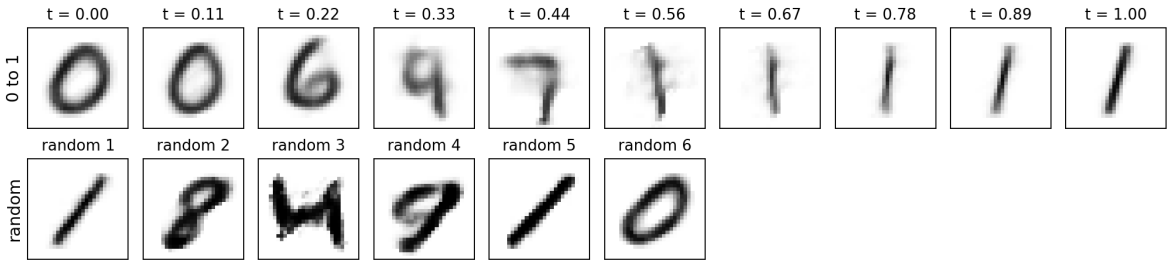


Figure 48: Decoded latent points of the autoencoder with $d = 2$, marked in Figure 46. Top, ten points on the straight line from the code of a typical 0 ($t = 0$) to the code of a typical 1 ($t = 1$). Bottom, six codes drawn uniformly from the box of the training codes.

and a 7, then to faint, smeared strokes, and finally to 1s. The line crosses the regions of other digits, and in its middle it runs through a sparse part of the cloud, where the nearest training code lies up to 6.7 times farther away than the typical spacing between neighbouring training codes, defined as the median distance from a training code to its nearest neighbour among the other training codes, and there the images are faint and blurred. Because the decoder is a continuous function, points between nearby codes of the same digit usually decode to blurry blends of their neighbours. Nothing in the training, however, guarantees that the image at a given code is a plausible digit, and the region between the clusters is where this guarantee is missing.

The second choice is a code drawn at random, which requires a distribution to draw it from. The autoencoder supplies none, since its codes have no prescribed mean, spread or shape. A distribution chosen without guidance from the training, here uniform over the box of the training codes, gives mixed results. The random codes 1, 5 and 6 of Figure 46 fall inside or at the edge of the cloud and decode to two 1s and a 0. Codes 2, 3 and 4 lie far outside, up to 157 typical spacings from the nearest training code, and decode to a distorted 8, a 9 and a shape that is no digit. In two dimensions about half of the box holds codes, and the fraction shrinks with the dimension of the latent space. Suppose that the codes occupy a fraction ρ of the range of each latent coordinate, independently of the others, as in Figure 49. They then fill a fraction ρ^d of the box. The measured 53.3% in two dimensions corresponds to $\rho = \sqrt{0.533} \approx 0.73$, and with the same ρ in sixteen dimensions

$$\rho^{16} \approx 0.73^{16} \approx 0.0065, \tag{8.2}$$

so that more than 99% of the random codes of a 16-dimensional latent space would fall where no training sample was ever encoded.

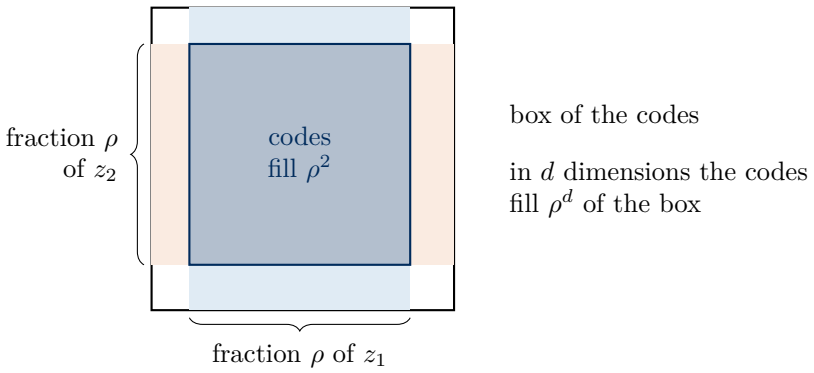


Figure 49: The empty part of the latent box grows with the dimension. If the codes occupy a fraction ρ of the range of each coordinate, independently, they fill ρ^2 of the box in two dimensions and ρ^d in d dimensions, equation (8.2). Schematic.

The same deficiency appeared from the side of anomaly detection. The reconstruction error of a platelet was small although no platelet had been seen, because the autoencoder measures how well a sample can be rebuilt and assigns no probability to it. In both cases what is missing is a probability distribution. The autoencoder maps the data to a scattered set of points and back. It compresses and reconstructs, but its latent space carries no probability structure, neither a density of the codes from which new codes could be drawn nor a density of the data with which a sample could be judged improbable. A model that generates new samples needs codes distributed according to a known density $p(\mathbf{z})$, and a decoder trained so that the codes drawn from that density, including those between the codes of the training

samples, decode to plausible samples. Such a model defines a distribution of the data,

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, \mathrm{d}\mathbf{z}, \tag{8.3}$$

in which the conditional density $p(\mathbf{x} \mid \mathbf{z})$, given by the decoder, describes how a sample arises from its code. It answers the two questions that the autoencoder left open, how to draw new samples and how probable a given sample is.

Symbols

The table lists the symbols of the chapter with the sections in which they are introduced.

symbol	meaning
$\mathbf{x}_i$, x_{ij}	sample i and its feature j (throughout)
n, D	number of samples, number of features of a sample (throughout)
X, X_c	data matrix with rows $\mathbf{x}_i^\top$, centred data matrix (Sections 1 and 2)
$p(\mathbf{x})$	probability density of the data (Sections 1 and 8)
η	standard deviation of the coordinates of random points (Section 1.1)
$\bar{\mathbf{x}}$	mean of the samples (Section 2)
C	covariance matrix of the features (Section 2)
$\mathbf{v}_l$, λ_l	principal direction l and its variance, an eigenvector and eigenvalue of C (Section 2)
V, V_d, Λ	matrix of all principal directions, of the first d, diagonal matrix of eigenvalues (Section 2)
$\mathbf{u}$, t_i	unit vector and the coordinates of the samples along it (Section 2.1); $\mathbf{u}_l$ left singular vectors (Section 2.3)
$w_i^{(l)}$, $\mathbf{w}_i$	PCA score of sample i on component l, vector of its first d scores (Section 2)
$\hat{\mathbf{x}}$	reconstruction of a sample, by PCA (Section 2) or by an autoencoder (Section 5)
d	dimension of the reduced representation, number of principal components, dimension of a map or of the latent space
R, $\mathbf{r}_a$, c_l	orthonormal basis of a d-dimensional plane, its columns, and the weights in equation (2.13) (Section 2.2)
$\mathcal{E}(R)$	mean squared error of projection onto the plane of R (Section 2.2)
r_d	explained-variance ratio of d components (Section 2.2)
U, S, s_l, m	factors of the singular value decomposition, singular values, $m = \min(n, D)$ (Section 2.3)
X_d	best rank-d approximation of the data matrix (Section 2.3)
$\|\cdot\|_F$	Frobenius norm of a matrix (Section 2.3)
F, F_d, ς_l, $\mathbf{e}_l$, $\mathbf{f}_l$	general matrix of the Eckart–Young theorem, its best rank-d approximation, singular values and singular vectors (Section 2.3)
χ_D	chi-distributed variable with D degrees of freedom (Section 1.1)
$\mathbb{1}$, $\mathbb{1}_d$	identity matrix, with explicit size d when needed
$\mathbf{y}_i$	point of the t-SNE map representing sample i (Section 3)
$p_{j\mid i}$, p_{ij}, P	conditional and joint neighbour probabilities in the data and their distribution over pairs (Section 3)
q_{ij}, Q	neighbour probabilities in the t-SNE map and their distribution over pairs (Section 3)
σ_i	width of the Gaussian around sample i (Section 3)
H_i, Perp	entropy of the neighbour distribution, perplexity (Section 3)
$\mathcal{C}$, E_{ij}, Z	t-SNE objective, Student-t kernel and its normalisation (Section 3)
k, S_c, $\boldsymbol{\mu}_c$	number of clusters, set of samples in cluster c, its centre (Section 4)

Continued on the next page

symbol	meaning (continued)
J	k-means objective, the within-cluster sum of squares (Section 4)
N_{gc}	contingency table, samples with label g in cluster c (Section 4.2)
N_2, T_b, T_l, T_c	numbers of pairs: all, together in both partitions, together by label, together by cluster (Section 4.2)
RI, ARI	Rand index, adjusted Rand index (Section 4.2)
sil_i, $\bar{\delta}_i^{\mathrm{in}}$, $\bar{\delta}_i^{\mathrm{out}}$	silhouette of sample i and its mean distances to its own and to the nearest other cluster (Section 4.3)
ε, $n_{\min}$	radius and minimum number of samples of DBSCAN (Section 4.5)
$f_{\boldsymbol{\phi}}$, $g_{\boldsymbol{\theta}}$	encoder and decoder with parameters $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ (Section 5)
$\mathbf{z}$, $\mathbf{z}_i$	latent code of a sample (Sections 5 and 8)
$\mathcal{L}$	reconstruction loss of an autoencoder (Section 5)
W_k, $\mathbf{b}_k$, $\mathbf{h}_k$	weights, biases and hidden activations of the dense autoencoder (Section 5.2)
A, B, G, M	encoder and decoder matrices of a linear autoencoder, an invertible $d \times d$ matrix, $M = BA$ (Section 5.3)
$a(\mathbf{x})$, τ	anomaly score, threshold (Section 7)
N_A, N_N	numbers of anomalies and normal samples in a test set (Section 7.3)
TP, FP, FN, TN	entries of the confusion matrix (Section 7.3)
TPR, FPR, AUC	true-positive rate, false-positive rate, area under the ROC curve (Section 7.3)
$p_\mathrm{N}(a)$, $p_\mathrm{A}(a)$	densities of the score for normal samples and anomalies (Section 7.3)
Φ	standard normal distribution function (Section 7.5)
π	fraction of anomalies in the data, the prevalence (Section 7.6)
ρ	fraction of the range of a latent coordinate filled by the codes (Section 8)

References

[1] K. Beyer, J. Goldstein, R. Ramakrishnan and U. Shaft, When is "nearest neighbor" meaningful?, in *Database Theory — ICDT'99*, Lect. Notes Comput. Sci. **1540**, 217 (1999), doi:10.1007/3-540-49257-7_15.

[2] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE **86**, 2278 (1998), doi:10.1109/5.726791.

[3] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister and B. Ni, MedMNIST v2 — a large-scale lightweight benchmark for 2D and 3D biomedical image classification, Sci. Data **10**, 41 (2023), doi:10.1038/s41597-022-01721-8.

[4] A. Acevedo, A. Merino, S. Alférez, Á. Molina, L. Boldú and J. Rodellar, A dataset of microscopic peripheral blood cell images for development of automatic recognition systems, Data Brief **30**, 105474 (2020), doi:10.1016/j.dib.2020.105474.

[5] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, Calibrating probability with undersampling for unbalanced classification, IEEE Symposium Series on Computational Intelligence, 159 (2015), doi:10.1109/SSCI.2015.33; data `https://www.openml.org/d/1597`.

[6] J. N. Weinstein *et al.*, The Cancer Genome Atlas Pan-Cancer analysis project, Nat. Genet. **45**, 1113 (2013), doi:10.1038/ng.2764.

[7] S. Fiorini, gene expression cancer RNA-Seq, UCI Machine Learning Repository (2016), doi:10.24432/C5R88H.

[8] K. Pearson, On lines and planes of closest fit to systems of points in space, Phil. Mag. **2**, 559 (1901), doi:10.1080/14786440109462720.

[9] H. Hotelling, Analysis of a complex of statistical variables into principal components, J. Educ. Psychol. **24**, 417 (1933), doi:10.1037/h0071325.

[10] C. Eckart and G. Young, The approximation of one matrix by another of lower rank, Psychometrika **1**, 211 (1936), doi:10.1007/BF02288367.

[11] L. van der Maaten and G. Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. **9**, 2579 (2008), https://jmlr.org/papers/v9/vandermaaten08a.html.

[12] G. E. Hinton and S. T. Roweis, Stochastic neighbor embedding, Advances in Neural Information Processing Systems **15** (2002).

[13] M. Wattenberg, F. Viégas and I. Johnson, How to use t-SNE effectively, Distill (2016), doi:10.23915/distill.00002.

[14] S. P. Lloyd, Least squares quantization in PCM, IEEE Trans. Inf. Theory **28**, 129 (1982), doi:10.1109/TIT.1982.1056489.

[15] D. Arthur and S. Vassilvitskii, k-means++: the advantages of careful seeding, Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 1027 (2007).

[16] W. M. Rand, Objective criteria for the evaluation of clustering methods, J. Am. Stat. Assoc. **66**, 846 (1971), doi:10.1080/01621459.1971.10482356.

[17] L. Hubert and P. Arabie, Comparing partitions, J. Classif. **2**, 193 (1985), doi:10.1007/BF01908075.

[18] P. J. Rousseeuw, Silhouettes: a graphical aid to the interpretation and validation of cluster analysis, J. Comput. Appl. Math. **20**, 53 (1987), doi:10.1016/0377-0427(87)90125-7.

[19] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), 226 (1996).

[20] M. A. Kramer, Nonlinear principal component analysis using autoassociative neural networks, AIChE J. **37**, 233 (1991), doi:10.1002/aic.690370209.

[21] G. E. Hinton and R. R. Salakhutdinov, Reducing the dimensionality of data with neural networks, Science **313**, 504 (2006), doi:10.1126/science.1127647.

[22] P. Baldi and K. Hornik, Neural networks and principal component analysis: learning from examples without local minima, Neural Netw. **2**, 53 (1989), doi:10.1016/0893-6080(89)90014-2.

[23] H. Bourlard and Y. Kamp, Auto-association by multilayer perceptrons and singular value decomposition, Biol. Cybern. **59**, 291 (1988), doi:10.1007/BF00332918.

[24] A. Krizhevsky, Learning multiple layers of features from tiny images, technical report, University of Toronto (2009), https://www.cs.toronto.edu/~kriz/cifar.html.

[25] T. Fawcett, An introduction to ROC analysis, Pattern Recognit. Lett. **27**, 861 (2006), doi:10.1016/j.patrec.2005.10.010.

[26] J. A. Hanley and B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology **143**, 29 (1982), doi:10.1148/radiology.143.1.7063747.